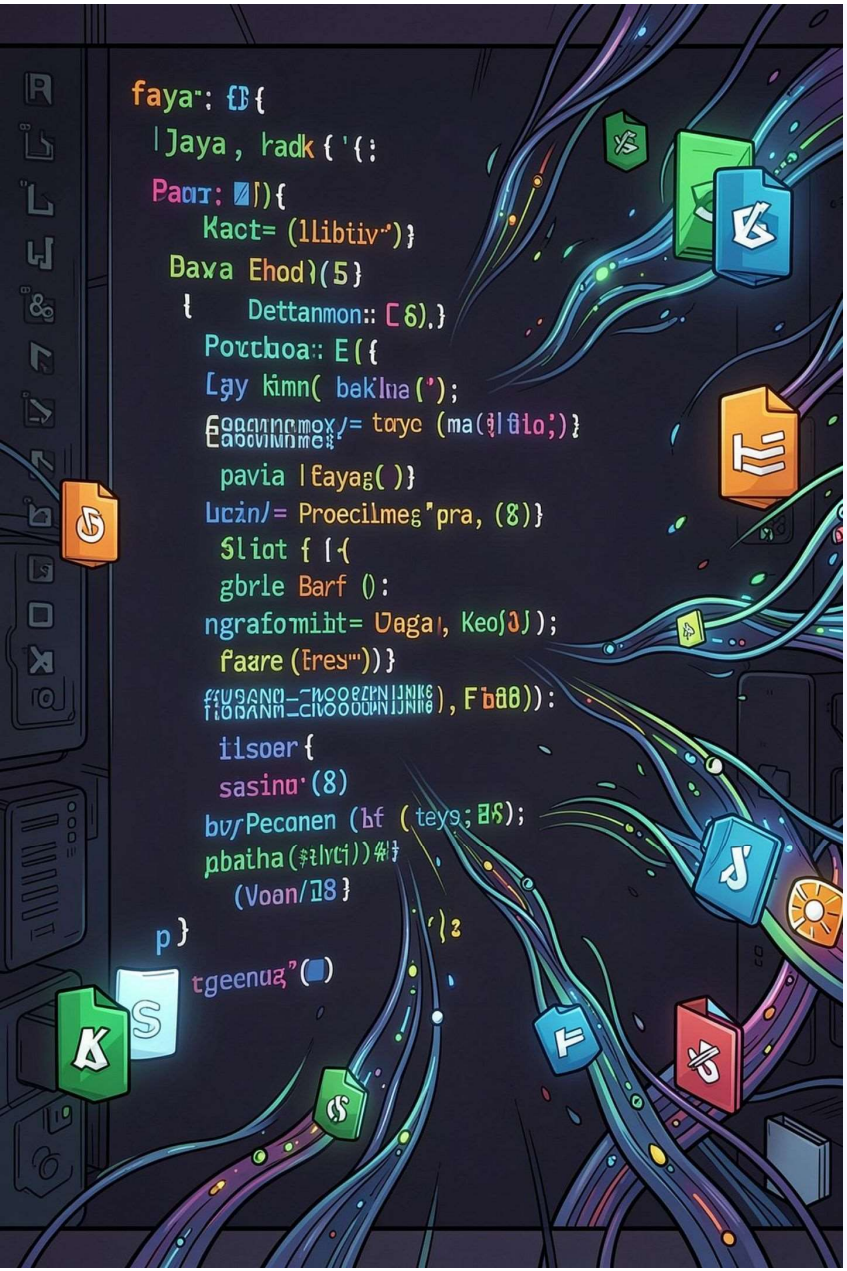


Gestión de Ficheros Java

De Bytes a XML

Un recorrido completo por el ecosistema de entrada/salida en Java: desde el manejo básico de archivos hasta el procesamiento de documentos XML estructurados.

Los datos de un programa permanecen en la memoria RAM del ordenador, lo que significa que en el momento en que la aplicación finalice o la máquina se apague, toda esta información desaparecerá. Veremos cómo almacenar información de forma permanente en ficheros, así como su posterior recuperación.

Java dispone de un gran número de clases incluidas en el paquete **java.io** específicamente creadas para tratar con ficheros de distintos tipos y manipular información en diferentes formatos. Estudiaremos algunas de estas clases, centrando nuestro análisis en el almacenamiento y recuperación de información en disco en forma de bytes, tipos básicos, texto y objetos.



INFORMACIÓN SOBRE FICHEROS Y DIRECTORIOS.

Clase File

Los ficheros o archivos y directorios del disco se representan de forma lógica en las aplicaciones Java como objetos de la clase File.

La clase File no se utiliza para transferir datos entre la aplicación y el disco, sino para obtener información sobre los ficheros y directorios de éste e incluso para la creación y eliminación de los mismos.

Creación de un objeto File

Existen diversas formas de crear un objeto File en función de cómo se indique la localización del fichero o directorio que va a representar. Por ejemplo, el constructor *File(String path)* permite **construir un objeto File a partir de su dirección absoluta o relativa al directorio actual**:

```
File f = new File ("datos.txt");
File f2 = new File ("misubdirectorio");
```

La clase File tiene tres constructores:

```
File(String path)
File(String path, String name)
File(File dir, String name)
```

Ejemplos de uso de la clase **File** donde se muestran diversas formas para declarar un fichero o directorio:

```
File fichero1 = new File (“C:\\dir\\file.txt”); //Windows
File fichero1 = new File (“/home/dir/file.txt”); //Linux
String directorio = “/home/dir”;
File fichero2 = new File (directorio, “fichero.txt”);
File direc = new File (directorio);
File fichero3 = new File (direc, “fichero.txt”);
```

La creación del objeto no implica que exista el fichero o directorio indicado en la ruta. Si éste no existe, las anteriores instrucciones no provocarán ninguna excepción, aunque tampoco será creado de forma implícita.

Para crear físicamente el fichero o directorio indicado en el constructor de File habrá que recurrir a los siguientes métodos de la clase:

- **boolean createNewFile()**. Crea el fichero cuyo nombre ha sido especificado en el constructor. Devuelve *true* si se ha podido crear el fichero, mientras que el resultado será *false* si ya existía y por tanto no ha sido creado.

Por ejemplo, si el fichero datos.txt del ejemplo anterior no existe, se podría ejecutar la siguiente instrucción para crearlo:

```
f.createNewFile();
```

La llamada a *createNewFile()* puede provocar una excepción IOException que habrá que capturar.

Si un objeto File hace referencia a un fichero no existente y no es creado de forma explícita invocando a *createNewFile(),* la creación del mismo se llevará a cabo implícitamente cuando se vaya a utilizar el objeto File para construir un objeto Writer u OutputStream, a fin de realizar una operación de escritura sobre el fichero.

- **boolean mkdir()**. Crea el directorio cuyo nombre ha sido especificado en el constructor. Si el directorio no existía y se ha podido crear, el método devolverá *true,* de lo contrario el resultado será *false.*

Por ejemplo, para crear físicamente el subdirectorio referenciado por el objeto File f2 sería:

```
f2.mkdir();
```

La clase File también proporciona un constructor que permite crear un objeto File asociado a un fichero, indicando a través de otro objeto File el directorio donde se encuentra dicho fichero:

File (File dir, String nombre_fichero)

En este caso, el directorio especificado en *dir* debe existir, de lo contrario se producirá una excepción al intentar utilizar el fichero para realizar cualquier operación sobre el mismo.

El siguiente bloque de instrucciones crearía un fichero llamado info.txt en el interior del directorio *datos:*

```
File f=new File ("datos");
f.mkdir();
//si datos no existiese y esta instrucción no estuviera, la llamada a createNewFile() provocaría una excepción.
File fichero=new File (f, "info.txt");
fichero.createNewFile();
```

Información sobre un fichero/directorio

Una vez creado el objeto File asociado al fichero o directorio, podemos obtener información del mismo aplicándole los siguientes métodos de la clase File:

- **boolean canRead()**. Indica si se puede o no leer el fichero.
- **boolean canWrite()**. Indica si se puede o no escribir en el fichero.
- **boolean canExecute()**. Indica si se puede o no ejecutar el fichero.
- **setExecutable(boolean executable)**. Permite cambiar los permisos de ejecución del fichero.
- **setExecutable(boolean executable, boolean ownerOnly)**
- **setReadable(boolean readable)**. Permite cambiar los permisos de lectura del fichero.
- **setReadable(boolean readable, boolean ownerOnly)**
- **setWritable(boolean writable)**. Permite cambiar los permisos de escritura del fichero.
- **setWritable (boolean writable, boolean ownerOnly)**
- **boolean exists()**. Indica si existe o no el fichero o directorio indicado en la ruta.
- **boolean isFile()**. Indica si el objeto File hace referencia o no a un fichero.
- **boolean isDirectory()**. Indica si el objeto File hace referencia o no a un directorio.

- **boolean isHidden()**. Comprobar si un fichero o directorio está oculto.
- **String getName()**. Devuelve el nombre del fichero sin el path.
- **String getAbsolutePath()**. Devuelve el path absoluto completo.
- **String getPath()**. Devuelve el path relativo.
- **String getParent()**. Devuelve el nombre del directorio padre o null si no existe.
- **long lastModified()**. Devuelve la última vez que el fichero ha sido modificado en milisegundos.
- **long length()**. Devuelve el tamaño del fichero en bytes.
- **String[] list()**. Devuelve un array de String con los nombres de los ficheros y directorios
- **String[] list(FilenameFilter filter)**. Devuelve un array de String con los nombres de los ficheros y directorios que cumplen el filtro pasado como argumento.

Los métodos **list()** y **listFiles()** listan el contenido de un directorio: **list()** devuelve un vector de String con los nombres de los archivos; **listFiles()** devuelve un vector de objetos File. Es posible listar los nombres de archivo de la raíz del sistema de archivos, mediante el método estático listRoots().

Eliminación y renombrado

Para eliminar físicamente un fichero o directorio la clase File proporciona el método *delete*() con el siguiente formato:

- **boolean delete()**. Elimina el fichero o directorio especificado por el objeto. Si el elemento ha podido ser eliminado el método devolverá *true,* si no devolverá *false.* Un directorio sólo podrá ser eliminado si está vacío; si no lo está, la llamada a *delete*() sobre el objeto File devolverá *false,* aunque no provocará ninguna excepción.

Si lo que queremos es renombrar un fichero o directorio existente, utilizaríamos el siguiente método:

- **boolean renameTo(File nuevo)**. Renombra el fichero o directorio, asignándole el nombre del objeto File especificado. La llamada a este método devolverá *true* si se ha podido renombrar el elemento, mientras que el resultado será *false* si esto no ha sido posible. En el caso de un directorio, no es necesario que esté vacío para poderlo renombrar.

El siguiente ejemplo renombra un fichero existente:

```
File f=new File (“fichero.txt”);
f.createNewFile(); //para poderlo renombrar debe existir
File f2=new File (“nuevo_nombre.txt”);
f.renameTo (f2) ; //renombra “fichero.txt” a ”nuevo_nombre.txt”
```

Establecer la fecha y la hora de modificación del archivo con **setLastModified()**. Por ejemplo, se podría hacer:

```
new File(“prueba.txt”).setLastModified(new Date().getTime());
```

para establecerle la fecha actual al fichero que se le pasa como parámetro, en este caso *prueba.txt*.

EJERCICIOS

1.- Programa que muestra todas las entradas del directorio “c:\windows” o "/home/usuario"

2.- Programa que diga cuál es el directorio actual.

3.- Hacer un programa que liste el contenido del directorio actual y de los directorios que cuelgan de él, mostrando nombre y tipo de cada fichero o directorio.

4.- Realiza un programa que cree la estructura de directorios y ficheros siguiente:

<D:\Home>

<d1>

|-------- f11.txt

<d2>

|-------- f21.txt

5.- Cambia después el nombre de f11.txt por f12.txt, mueve el fichero f21.txt al directorio d1 y eliminando después el directorio d2.

6.- Escribe un programa que muestre los atributos de un fichero o directorio cuyo nombre se solicita al usuario.

Nombre del archivo Camino absoluto Tamaño
Si es Fichero
Si es Directorio
Última modificación Si tiene permiso de escritura
Si tiene permiso de lectura
Si tiene permiso de ejecución

7.- Realiza un programa que compruebe si se tiene acceso de ejecución y lectura a un fichero (pasado como argumento) y después modifique sus atributos denegando estos permisos **(No funciona en Windows)**.

8- Realiza un programa que cree una estructura de directorios como la siguiente, donde fx son ficheros y dx son directorios,

/home/

/usr1

f1

/usr2

/d1

f2

/d2

f3

f4

/d3

9- Escribir un programa que liste el contenido de un directorio pasado como argumento, de cada elemento debe mostrar: tipo (fichero o directorio)., nombre y si es fichero, el tamaño.

10- Crear un programa en Java que borre la estructura anterior, solicitando confirmación antes de borrar cada fichero o directorio y sin borrar ningún directorio que no esté vacío (cuidado con el directorio elegido).

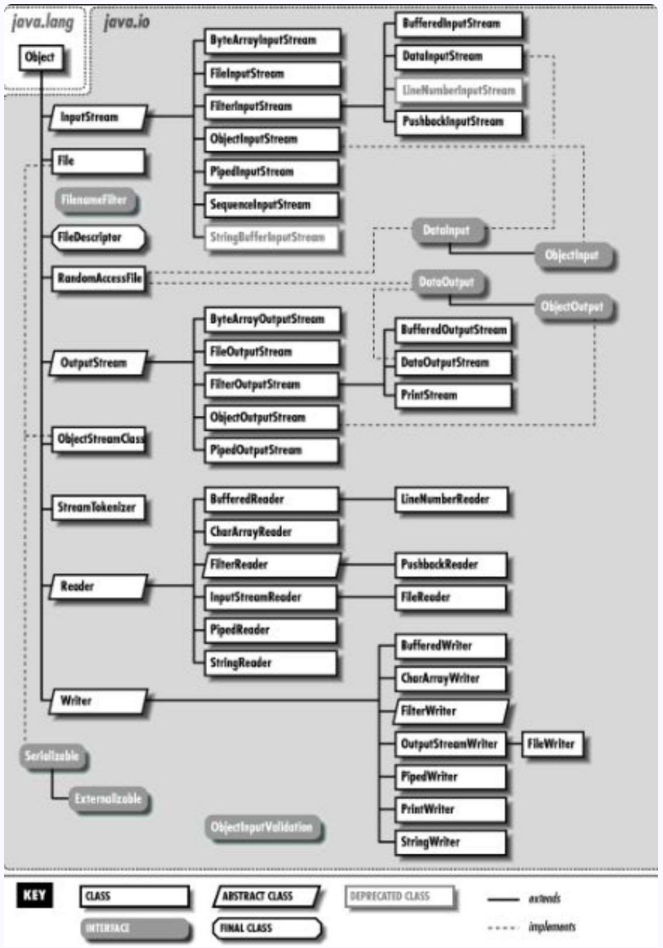
El Corazón de Java I/O

La librería **java.io** contiene las clases necesarias para gestionar las operaciones de entrada y salida con Java. Se basa en flujos de E/S (E/S Stream), un poderoso concepto que simplifica enormemente las operaciones de E / S. Más adelante, analizaremos la serialización, que permite que un programa escriba objetos completos en flujos y los vuelva a leer y las operaciones de E / S de archivos y del sistema de archivos, *incluidos los archivos de acceso aleatorio*.

La librería java.io Estas clases de E/S las podemos agrupar fundamentalmente en:

- Clases para leer entradas desde un flujo de datos.
- Clases para escribir entradas a un flujo de datos.
- Clases para operar con ficheros en el sistema de ficheros local.
- Clases para gestionar la serialización de objetos.

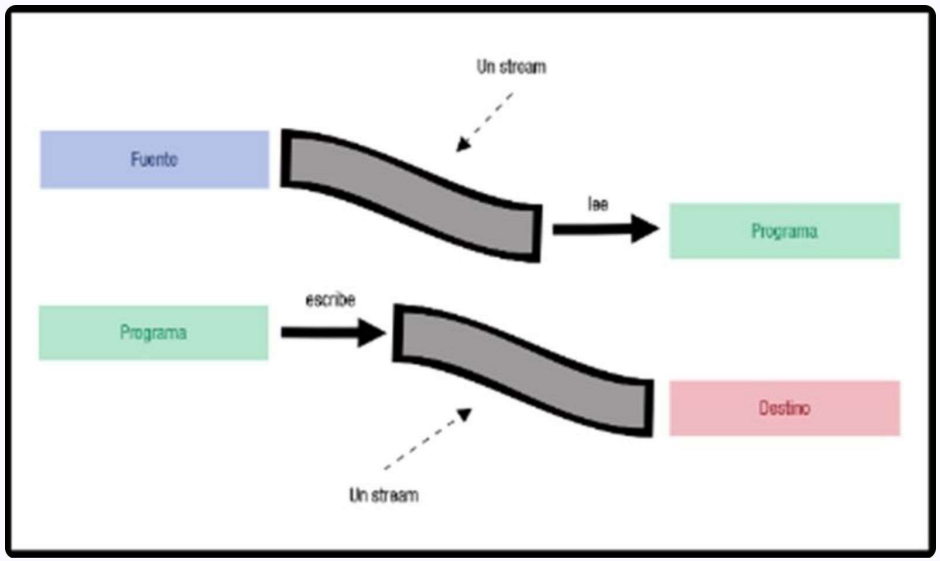
En la imagen puedes ver las clases de las que se dispone en **java.io**:



Flujos de datos

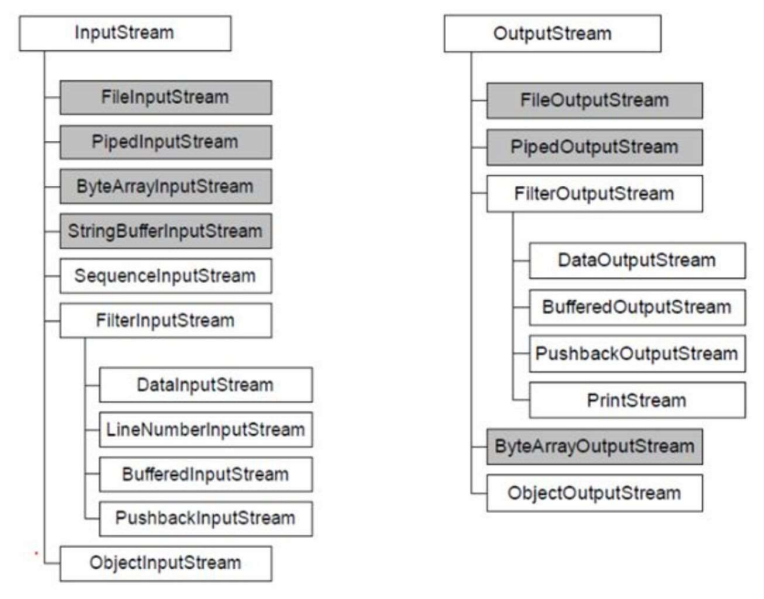
Un **flujo (stream)** es una **abstracción de todo aquello que produce o consume información**. **La vinculación de este flujo al dispositivo físico la hace el sistema de entrada y salida de Java**.

Un stream es una conexión entre el programa y la fuente o destino de los datos. La información se traslada en serie (un carácter a continuación de otro) a través de esta conexión. Esto da lugar a una forma general de representar muchos tipos de comunicaciones.



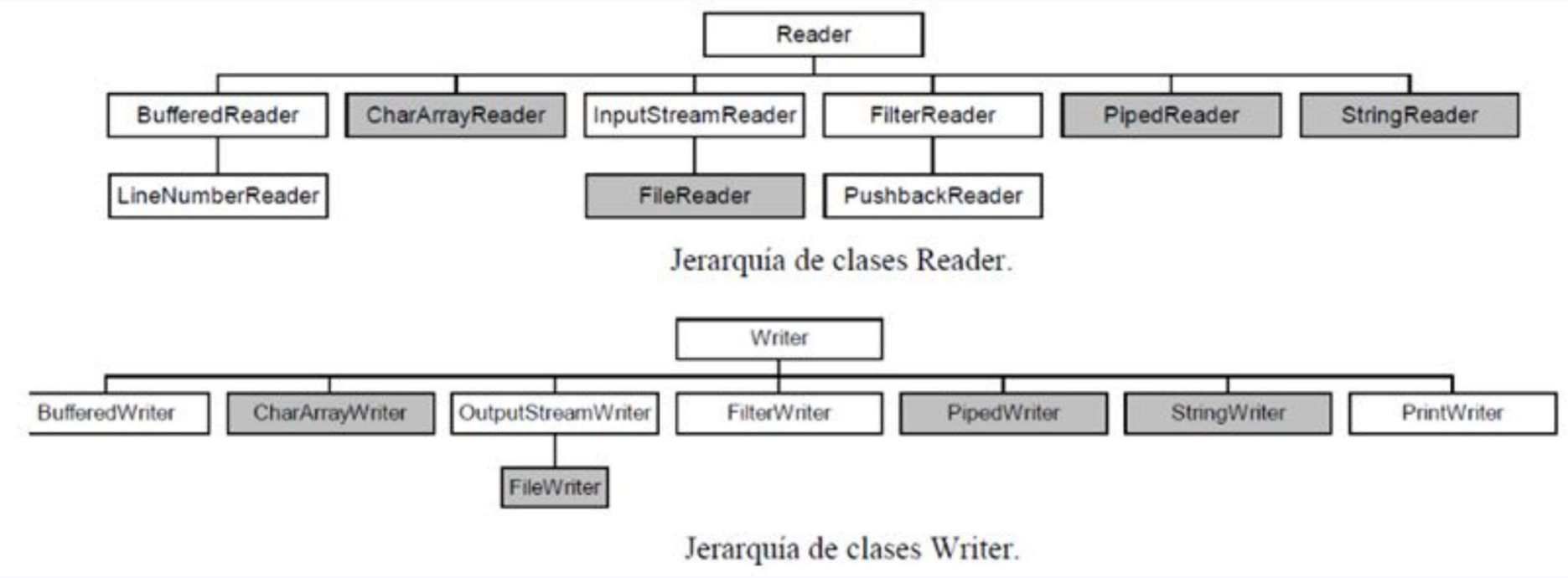
Las clases y métodos de E/S que necesitamos emplear son las mismas independientemente del dispositivo con el que estemos actuando. Luego, el núcleo de Java sabrá si tiene que tratar con el teclado, el monitor, un sistema de archivos o un socket de red; liberando al programador de tener que saber con quién está interactuando.

Existen dos tipos de flujos, flujos de bytes (byte streams) y flujos de caracteres (character streams).



Los **flujos de bytes** (8 bits) se usan para manipular datos binarios, legibles solo por la máquina (por ejemplo, un fichero de programa). Su uso está orientado a la lectura y escritura de datos binarios (imágenes, PDFs, audio...). Trabajan con bytes puros sin codificación.

El tratamiento del flujo de bytes viene determinado por dos clases abstractas que son **InputStream** y **OutputStream**. Estas dos clases son las que definen los métodos que sus subclases tendrán implementados y, de entre todos, destacan read() y write() que leen y escriben bytes de datos respectivamente.



Jerarquía de clases Reader.

Jerarquía de clases Writer.

- Los **flujos de caracteres** (16 bits) se usan para manipular datos legibles por humanos (por ejemplo, un fichero de texto). Vienen determinados por dos clases abstractas: **Reader** y **Writer**. Dichas clases manejan flujos de caracteres Unicode. De ellas derivan subclases concretas que implementan los métodos definidos destacados los métodos read() y write() que, en este caso, leen y escriben caracteres de datos respectivamente.

Destacamos las clases del paquete java.io relativas a flujos:

- **BufferedInputStream**: permite leer datos a través de un flujo con un buffer intermedio.
- **BufferedOutputStream**: implementa los métodos para escribir en un flujo a través de un buffer.
- **FileInputStream**: permite leer bytes de un fichero.
- **FileOutputStream**: permite escribir bytes en un fichero o descriptor.
- **StreamTokenizer**: esta clase recibe un flujo de entrada, lo analiza (parse) y divide en diversos pedazos (tokens), permitiendo leer uno en cada momento.
- **StringReader**: es un flujo de caracteres cuya fuente es una cadena de caracteres o string.
- **StringWriter**: es un flujo de caracteres cuya salida es un buffer de cadena de caracteres, que puede utilizarse para construir un string.

Destacar que hay clases que se **"montan" sobre otros flujos para modificar la forma de trabajar con ellos**. Por ejemplo, con **BufferedInputStream** podemos añadir un buffer a un flujo **FileInputStream**, de manera que se mejore la eficiencia de los accesos a los dispositivos en los que se almacena el fichero con el que conecta el flujo.

Las clases orientadas al flujo de bytes nos proporcionan la suficiente funcionalidad para realizar cualquier tipo de operación de entrada o salida, pero no pueden trabajar directamente con **caracteres Unicode**, los cuales están **representados por dos bytes**. Por eso, se consideró necesaria la creación de las clases orientadas al flujo de caracteres para ofrecernos el soporte necesario para el tratamiento de caracteres.

Para los flujos de caracteres, Java dispone de dos clases abstractas: **Reader** y **Writer**. Reader, Writer, y todas sus subclases, reciben en el constructor el objeto que representa el flujo de datos para el dispositivo de entrada o salida.

Hay que recordar que **cada vez que se llama a un constructor se abre el flujo de datos y es necesario cerrarlo** cuando no lo necesitemos.

Existen muchos tipos de flujos dependiendo de la utilidad que le vayamos a dar a los datos que extraemos de los dispositivos.

Un flujo puede ser envuelto por otro flujo para tratar el flujo de datos de forma cómoda. Así, un **bufferWriter** nos permite manipular el flujo de datos como un buffer, pero si lo envolvemos en un **PrintWriter** lo podemos escribir con muchas más funcionalidades adicionales para diferentes tipos de datos.

En este ejemplo de código, se ve cómo podemos escribir la salida estándar a un fichero. Cuando se teclee la palabra "salir", se dejará de leer y entonces se saldrá del bucle de lectura.

Podemos ver cómo se usa **InputStreamReader** que es un puente de flujos de bytes a flujos de caracteres: lee bytes y los decodifica a caracteres. **BufferedReader** lee texto de un flujo de entrada de caracteres, permitiendo efectuar una lectura eficiente de caracteres, vectores y líneas.

Como vemos en el código, usamos **FileWriter** para flujos de caracteres, pues para datos binarios se utiliza **FileOutputStream**.

```
try {
   PrintWriter out = null;
   out = new PrintWriter(new FileWriter("c:\\salida.txt", true));
   BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
   String s;
   while (!(s = br.readLine()).equals("salir")){
      out.println(s);
   }
   out.close();
}
catch(IOException ex){
   System.out.println(ex.getMessage());
}
```

Rutas de los ficheros

En los ejemplos que vemos en el tema estamos usando la ruta de los ficheros tal y como se usan en Linux o Windows. Cuando operamos con rutas de ficheros, el carácter separador entre directorios o carpetas suele cambiar dependiendo del sistema operativo en el que se esté ejecutando el programa.

Para evitar problemas en la ejecución de los programas cuando se ejecuten en uno u otro sistema operativo, y por tanto persiguiendo que nuestras aplicaciones sean lo más portables posibles, se recomienda usar en Java: **File.separator**.

Podríamos hacer una función que al pasarle una ruta nos devolviera la adecuada según el separador del sistema actual, del siguiente modo:

```
import java.io.File;
import java.io.IOException;

public class ConversorRutas {
    public static void main(String[] args) throws IOException {
        // 1. Ruta base simulada usando un array de carpetas para construirla de forma limpia
        String[] componentesRuta = { "usuarios", "documentos", "proyecto", "archivo.txt" };

        // 2. Construir la ruta usando File.separator
        StringBuilder rutaConstruida = new StringBuilder();
        for (int i = 0; i < componentesRuta.length; i++) {
            rutaConstruida.append(componentesRuta[i]);
            if (i < componentesRuta.length - 1) {
                rutaConstruida.append(File.separator);
            }
        }
        System.out.println("Ruta generada para el SO actual: " + rutaConstruida.toString());

        // 3. Crear el objeto File con la ruta adaptada
        File archivo = new File(rutaConstruida.toString());
        // archivo.createNewFile(); //Generación en disco

        // 4. Mostrar información del archivo/directorio
        System.out.println("Ruta absoluta: " + archivo.getAbsolutePath());

        // 5. Ejemplo de normalización si recibes una ruta con separadores incorrectos
        String rutaMixta = "carpeta/subcarpeta\\archivo2.txt";
        String rutaNormalizada = rutaMixta.replace("/", File.separator).replace("\\", File.separator);
        System.out.println("Ruta normalizada automáticamente: " + rutaNormalizada);
    }
```

Lectura y Escritura: Técnicas Eficientes

1

Buffer

`BufferedReader` y `BufferedWriter` reducen accesos a disco agrupando operaciones. Mejora drástica del rendimiento en ficheros grandes.

2

Scanner

Herramienta versátil para parsear tipos primitivos (`nextInt()`, `nextLine()`). Ideal para entrada estructurada y archivos de texto.

3

PrintStream

Volcado de texto con formato sencillo. Soporta `printf()` y escritura directa sin necesidad de gestionar excepciones de I/O.

Lectura de un fichero de texto

Además de abrir, operar y cerrar, debemos tener en cuenta también las clases Java a emplear, es decir, recuerda que hemos comentado que, si vamos a tratar con ficheros de texto, es más eficiente emplear las clases de Reader Writer, frente a las clases de InputStream y OutputStream que están indicadas para flujos de bytes.

Otra cosa a considerar, cuando se va a hacer uso de ficheros, es la forma de acceso al fichero que se va a utilizar, si va a ser de manera **secuencial** o bien **aleatoria**. En un fichero secuencial, para acceder a un dato debemos recorrer todo el fichero desde el principio hasta llegar a su posición. Sin embargo, en un fichero de acceso aleatorio podemos posicionarnos directamente en una posición del fichero, y ahí leer o escribir.

Distinguimos **dos tipos de ficheros: los de texto y los binarios**. En los ficheros de texto la información se guarda como caracteres. Esos caracteres están codificados en Unicode, o en ASCII u otras codificaciones de texto.

En la siguiente porción de código puedes ver cómo para un fichero existente, que en este caso es texto.txt, averiguamos la codificación que posee, usando el método **getEncoding().**

```
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.InputStreamReader;
import java.util.logging.Logger;
import java.util.logging.Level;

public class Pruebas {
    public static void main(String[] args) {
        FileInputStream fichero;
        try {
            // Elegimos fichero para leer flujos de bytes "crudos"
            fichero = new FileInputStream("F:\\materialAD\\Separator.java");
            // InputStreamReader sirve de puente de flujos de byte a caracteres
            InputStreamReader unReader = new InputStreamReader(fichero);
            // Vemos la codificación actual
            System.out.println(unReader.getEncoding());
        } catch (FileNotFoundException ex) {
            Logger.getLogger(Pruebas.class.getName()).log(Level.SEVERE, null, ex);
        }
    }
}
```

Aunque ya has visto un ejemplo que usa BufferedReader, insistimos aquí sobre la filosofía de estas clases, que usan la idea de un buffer. La idea es que cuando una aplicación necesita leer datos de un fichero, tiene que estar esperando a que el disco en el que está el fichero le proporcione la información. **Un dispositivo cualquiera de memoria masiva, por muy rápido que sea, es mucho más lento que la CPU del ordenador**.

Así que, **es fundamental reducir el número de accesos al fichero a fin de mejorar la eficiencia de la aplicación**, y para ello se asocia al fichero una memoria intermedia, el buffer, de modo que cuando se necesita leer un byte del archivo, en realidad se traen hasta el buffer asociado al flujo, ya que es una memoria mucho más rápida que cualquier otro dispositivo de memoria masiva.

Cualquier operación de Entrada/Salida a ficheros puede generar una **IOException**, es decir, un error de Entrada/Salida. Puede ser, por ejemplo, que el fichero no exista, o que el dispositivo no funcione correctamente, o que nuestra aplicación no tenga permisos de lectura o escritura sobre el fichero en cuestión. Por eso, las sentencias que involucran operaciones sobre ficheros, deben ir en un bloque try.

Muchas de las operaciones realizadas con ficheros en un programa tienen que ver con la lectura de ficheros de texto.

Para recuperar cadenas de caracteres de un fichero, el paquete java.io proporciona dos clases: **FileReader** y **BufferedReader**. A continuación, veremos cómo se utilizan estas clases, examinando los dos pasos que hay que seguir para la recuperación de cadenas de caracteres de un fichero.

Creación de un objeto FileReader

Para poder recuperar información de un fichero de texto, es necesario primeramente crear un objeto FileReader asociado al mismo. Un objeto FileReader representa un fichero de texto "abierto" para la lectura de datos. Este objeto es capaz de adaptar la información recuperada del fichero a las características de una aplicación Java, transformando los bytes almacenados en el mismo en caracteres unicode.

Se puede construir un objeto FileReader a partir de un objeto File existente o bien proporcionando directamente la ruta del fichero, a partir de los siguientes constructores:

FileReader (String path)

FileReader (File fichero)

La siguiente instrucción crearía un objeto FileReader que haga referencia al fichero datos.txt:

```
File f = new File ("datos.txt");
FileReader fr = new FileReader(f);
```

La clase FileReader proporciona el método *read()* para la lectura de la información almacenada en un fichero, resultando su uso bastante engorroso ya que los caracteres son recuperados como tipo byte, debiendo ser posteriormente convertidos a String. Es por ello que resulta más cómodo recurrir a la clase BufferedReader para realizar esta operación, utilizando el objeto FileReader como puente para crear un objeto de este tipo.

El siguiente fragmento de código muestra la lectura de un fichero carácter a carácter.

```
//Se debe importar java.io.FileReader
try {
   //Abrir el fichero indicado en la variable nombreFichero
   FileReader fr = new FileReader("prueba.txt");
  //Leer el primer carácter y almacenar en una variable de tipo int
   int caract = fr.read();
   //Se recorre el fichero hasta encontrar el carácter -1
   // que marca el final del fichero
   while(caract != -1) {
      //Mostrar en pantalla el carácter leído convertido a char
      System.out.print((char)caract);
      //Leer el siguiente carácter
      caract = fr.read();
   }
   //Cerrar el fichero
   fr.close();
} catch (Exception e) {
   System.out.println("Error de lectura del fichero");
}
```

Creación de un objeto BufferedReader

Es una buena práctica utilizar estas clases conjuntamente pues FileReader simplemente lee caracteres de un fichero y BufferedReader nos ayuda a guardarlos en un buffer para tratarlos de una forma más segura. Una vez creado el objeto FileReader, el segundo paso consiste en crear un BufferedReader, para lo cual utilizaremos el constructor:

```
BufferedReader (Reader entrada)
```

Para leer de un fichero el procedimiento es el mismo que cuando se utiliza para leer cadenas de caracteres desde el teclado, sólo que en este caso el objeto Reader que hay que proporcionar al constructor será el objeto FileReader asociado al fichero:

```
1º FileReader fr = new FileReader(f);
```

```
2º BufferedReader bf=new BufferedReader(fr);
```

Una vez creado el objeto, puede utilizarse el método ***readLine()*** para leer líneas de texto del fichero de forma similar a como se leían del teclado. En el caso de un fichero, y dado que éste puede estar formado por más de una línea, será necesario utilizar un bucle while para recuperar todas las líneas de texto del mismo de forma secuencial. El siguiente programa muestra por pantalla el contenido completo de un fichero llamado datos.txt:

```
import java.io.*;
public class Lectura {
   public static void main (String [] args)throws IOException {
      File f = new File ("datos.txt");
      if (f.exists()) {
         FileReader fr=new FileReader("datos.txt");
         BufferedReader bf=new BufferedReader (fr);
         String cad;
         while (( cad=bf.readLine())!=null )) {
            System.out.println(cad);
         }
      } else {
         System.out.println ("El fichero no existe") ;
      }
   }
}
```

Según se desprende del programa anterior, el método *readLine()* apunta a la siguiente línea de texto después de recuperar la línea actual. Cuando no existan más líneas para leer, la llamada a *readLine()* devolverá *null.*

Si lo que se quiere es leer carácter a carácter en vez de línea a línea, deberíamos utilizar el método *read()* en lugar de *readLine().* El método *read()* devuelve un entero que representa el código unicode del carácter leído, siendo el resultado -1 si no hay más caracteres para leer.

También es posible realizar la lectura de los datos almacenados en un fichero de texto utilizando la clase Scanner. Para ello, se pasará el objeto File asociado al fichero al constructor de Scanner; a partir de ahí, se puede hacer uso de los métodos de esta clase para recorrer el fichero y recuperar su contenido en forma de cualquier tipo primitivo Java.

```
import java.io.BufferedReader;
import java.io.File;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.IOException;
import java.util.Scanner;

public class Lectura {
   public static void main(String[] args) {
      print("\nScanner:\n");
      /*
      * La clase Scanner es la usada en el siguiente ejemplo, su ventaja es que es un poco mas sencilla.
      * Requiere un objeto File y un Objeto tipo Scanner.
      * Solo se podría lanzar una excepción cuando no se encuentra el archivo
      */
      File archivo=new File("c:/ejemplo.txt");
      try {
         Scanner lectura=new Scanner(archivo);
         while (lectura.hasNextLine()){
            System.out.println(lectura.nextLine());
         }
         lectura.close();
      } catch (FileNotFoundException e) {
         e.printStackTrace();
      }
```

```
        print("\nBufferedReader:\n");
        /*
        * El siguiente ejemplo está hecho con un BufferedReader.
        * Scanner puede leer enteros, chars, flotantes, decimales y líneas completas. Esa es la diferencia.
        * Se podría lanzar una excepción de archivo no encontrado y en dado caso un error de entrada salida,
        * puesto que es operación estrictamente I/O.
        */
        try {
           FileReader f=new FileReader("c:/ejemplo.txt");
           BufferedReader lectura = new BufferedReader(f);
           String linea=lectura.readLine();
           while(linea!=null) {
              System.out.println(linea);
              linea=lectura.readLine();
           }
           f.close();
        } catch (FileNotFoundException e) {
           e.printStackTrace();
        } catch (IOException e) {
           e.printStackTrace();
        }
     }
}
```

Escritura en ficheros de texto

La realización de esta operación también se lleva a cabo en dos pasos que implican la utilización de dos clases: **FileWriter** y **PrintWriter**. Ambas clases sirven para escribir caracteres en ficheros, simplemente PrintWriter es una mejora de la clase FileWriter (comparten el mismo padre java.io.Writer) que nos permite utilizar métodos adicionales.

Creación de un objeto FileWriter

Como primer paso para escribir en un fichero de texto, es necesario construir un objeto FileWriter que nos permita tener acceso al fichero en modo escritura. Para ello, podemos utilizar uno de los siguientes constructores:

- `FileWriter (String path)`
- `FileWriter (File fichero)`
- `FileWriter (String path, boolean append)`
- `FileWriter (File fichero, boolean append)`

Donde el parámetro ***append*** permite indicar si los datos que se van a escribir se añadirán a los ya existentes *(true)* o sobreescribirán a éstos *(false).* Si se utiliza uno de los dos primeros constructores, los datos escritos en el fichero sustituirán a los existentes (equivalente a *append=false*).

La clase **FileWriter** proporciona un método ***write()*** que permite escribir en el fichero la cadena de caracteres pasada como parámetro, aunque también puede utilizarse la clase estándar de escritura PrintWriter para realizar esta operación.

Creación del objeto PrintWriter

La utilización de la clase PrintWriter posibilita que la escritura sobre un fichero se realice de la misma forma que la escritura en pantalla. La diferencia está en que en el caso de la consola o pantalla el objeto PrintWriter asociado se encontraba en System.out, mientras que para un fichero de texto habrá que construir el objeto PrintWriter a partir de un objeto FileWriter:

```
1º FileWriter fw = new FileWriter("datos.txt");
```

```
2º PrintWriter out = new PrintWriter(fw);
```

Desde la versión Java 5 también es posible crear un objeto PrintWriter a partir de un objeto File o incluso de la ruta del fichero, por lo que las dos instrucciones anteriores podrían reducirse a una sola:

```
PrintWriter out = new PrintWriter ("datos.txt");
```

Una vez creado el objeto PrintWriter, podemos utilizar los métodos **print(), println() y printf()** para escribir en el fichero. El siguiente bloque de instrucciones almacenaría en un fichero el grupo de nombres existentes en un array:

```
import java.io.*;
public class Grabación {
   public static void main (String [] args) throws IOException {
      //array de nombres
      String [] nombres= {"ana", "rosa", "jorge", "manuel"} ;
      FileWriter fw = new FileWriter ("datos.txt");
      PrintWriter out=new PrintWriter (fw);
      for(int i=0;i<nombres.length;i++) {
         out.println (nombres[i]);
      }
      out.flush();
      out.close();
   }
}
```

Obsérvese la utilización de los métodos ***flush()*** y ***close()*** de PrintWriter después de la escritura en el fichero. La llamada al método *flush()* garantiza que todos los datos enviados a través del buffer de salida han sido escritos en el fichero, mientras que *close()* cierra la conexión con el fichero y libera los recursos utilizados por ésta.

EJERCICIOS

11. Escribe un programa que escriba los 100 primeros números naturales en un archivo numNaturales.txt.

12. Escribe un programa que reciba el nombre de un archivo que almacena una serie de cantidades enteras positivas e imprima su suma.

13. Escribir un método que reciba un array de cadenas de caracteres y vuelque su contenido a un archivo de nombre "*pruebaArrays.txt*". Las cadenas quedarán separadas en el archivo por un asterisco.

14. Escribir un método que reciba el archivo "*pruebaArrays.txt*" e imprima las cadenas que dicho archivo almacena (una cadena por línea). Se tendrá en cuenta que las distintas cadenas se separan en el archivo por un asterisco (que no se imprimirá).

15. Escribir un método que reciba el número correspondiente a un mes del año y la letra correspondiente al día de la semana en que cae el día 1 de ese mes ('L' para Lunes, 'M' para Martes, 'X' para Miércoles...). El método escribirá en un archivo un calendario para ese mes que estará formado por todos los días del mes seguidos de la letra correspondiente al día de la semana en que caen. El método comprobará que los valores recibidos son correctos y obtendrá el nombre concatenando a la palabra "mes" el número de mes recibido más la extensión ".txt".

Ejemplo: Si el método recibe el valor 3 (Marzo) y la letra 'M' indicando que el 1 de Marzo es Martes, obtendrá el nombre de archivo "mes3.txt" y lo escribirá con un calendario que tendrá un aspecto como éste: 1M2X3J5V6S7D8L...31J

16. Escribir un método que reciba un array de cadenas de caracteres y el nombre de un archivo. El método volcará el contenido del array de cadenas en el archivo especificado. Realizar otro método que muestre su contenido por pantalla.

17. Escribir un método que reciba el nombre de un archivo para dejar sus líneas ordenadas alfabéticamente (no distinguir minúsculas de mayúsculas a la hora de ordenar).

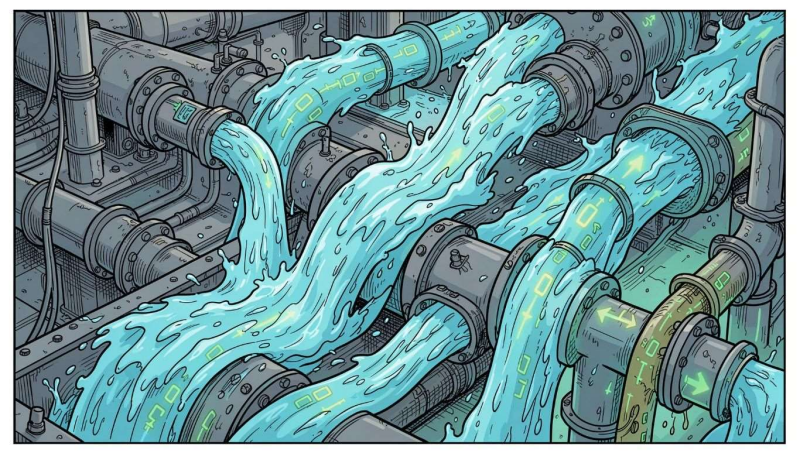
18. Escribir un método que solicite al usuario un valor entero. El método no dejará de solicitarlo hasta que el usuario introduzca un valor entero correcto, en ese caso el método devolverá dicho valor.

19. Escribir un método que solicite al usuario una respuesta a una pregunta. El método continuará formulando la pregunta hasta que la respuesta consista en uno de los siguientes caracteres: 's', 'S', 'n', 'N'. En caso de responder afirmativamente ('s' o 'S') el método devolverá el valor true, y false en caso contrario.

Escritura de datos primitivos en un fichero

Las clases **PrintWriter** y **BufferedReader** están pensadas para escribir y recuperar, respectivamente, **cadenas de caracteres en un fichero**.

Para **almacenar datos en un fichero en forma de tipos básicos Java**, el paquete java.io proporciona las clases **FileOutputStream** y **DataOutputStream**.



Creación de un objeto FileOutputStream

Este objeto es el **equivalente al FileWriter** utilizado en la escritura de cadenas de caracteres. Como en el caso de éste, se puede crear un objeto **FileOutputStream** que permita añadir información al fichero o sobrescribirla haciendo uso de los constructores:

FileOutputStream (File fichero, boolean append)

FileOutputStream (String path, boolean append)

La clase FileOutputStream es una subclase de OutputStream, la cual representa un stream o flujo de salida para la escritura de bytes.

Creación de un objeto DataOutputStream

A partir del objeto **FileOutputStream** se puede crear un objeto **DataOutputStream** para realizar la escritura de los datos. El constructor utilizado sería:

DataOutputStream (OutputStream)

La clase DataOutputStream proporciona métodos para escribir datos en un fichero en cada uno de los ocho tipos primitivos Java. Estos métodos tienen el formato:

void writeXxx(xxx dato)

siendo *xxx* el nombre del tipo primitivo Java.

El siguiente programa escribe en un fichero el contenido de un array de enteros:

```
import java.io.*;
public class GrabaArray {
   public static void main (String [] args)throws IOException{
      FileOutputStream fw= new FileOutputStream (“datos.mbd”,false);
      DataOutputStream ds=new DataOutputStream(fw);
      int [] m={5,10,3,6}; //array de enteros
      for(int i=0 ; i<m.length;i ++){
         ds.writeInt(m[i]);
      }
      ds.close(); //cierra el stream
   }
}
```

Lectura de datos primitivos desde un fichero

Para realizar la lectura de los datos almacenados en un fichero mediante DataOutputStream, el paquete java.io proporciona la clase **DataInputStream** que a su vez se apoya en **FileInputStream**.

Creación de un objeto FileInputStream

Es el equivalente al **FileReader** utilizado en la lectura de cadenas de caracteres. Se puede crear un objeto de esta clase a partir de un objeto File o de la cadena que representa la ruta del fichero.

FileInputStream es una subclase de InputStream que, al igual que OutputStream, está orientada al tratamiento de bytes. En este caso, a la lectura de bytes

Creación de un objeto DataInputStream

A partir del objeto **FileInputStream** se puede crear un **DataInputStream** para realizar la lectura de los datos.

Esta clase dispone de los métodos ***readXxx()*** para recuperar los datos almacenados en el tipo indicado, siendo *Xxx* el nombre del tipo primitivo.

El siguiente programa recupera todos los datos almacenados en el fichero del programa anterior y los muestra por pantalla:

```
import java.io.*;
public class VolcadoDatos {
   public static void main (String [] args) throws IOException {
      DataInputStream ds=new DataInputStream(new FileInputStream (“datos.mbd”));
      try {
         for (;;){ //bucle infinito
            System.out.println(ds.readInt());
         }
      } catch(EOFException e){
         ds.close(); //cierra el stream
      }
   }
}
```

Como se desprende del ejemplo anterior, el método *readInt()* devuelve el entero actual y tras la lectura de éste se posiciona en el siguiente dato entero del fichero. Cuando se llegue al final del fichero se producirá una excepción *EOFExcepction* al invocar al método *readInt(),* produciéndose una salida del bucle infinito y finalizando la aplicación.

Al ejecutar este programa se visualizará en pantalla:

```
5
10
3
```

Parsing Inteligente: StreamTokenizer

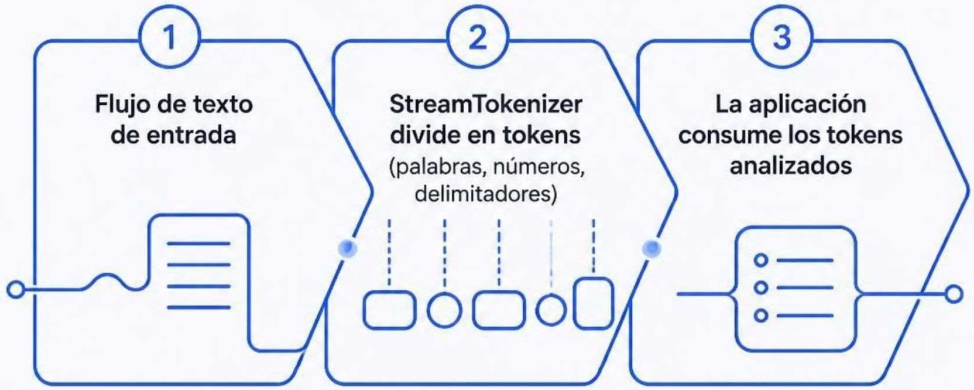
¿Qué hace?

La clase `StreamTokenizer` obtiene un flujo de entrada y lo divide en **tokens** significativos: palabras, números y delimitadores, de forma incremental.

Casos de uso ideales

- Procesamiento de archivos de configuración
- Interpretación de lenguajes simples
- Filtrado automático de comentarios y espacios

El ejemplo que puedes ver a continuación, muestra cómo utilizar la clase **StreamTokenizer** para contar números y palabras de un fichero de texto. Se abre el flujo con ayuda de la clase FileReader, y puedes ver cómo se "monta" el flujo StreamTokenizer sobre el FileReader, es decir, que se construye el objeto StreamTokenizer con el flujo FileReader como argumento, y entonces se empieza a iterar sobre él.



El método **nextToken** devuelve un int que indica el tipo de token leído. Hay una serie de constantes definidas para determinar el tipo de token:

- `TT_WORD indica que el token es una palabra.`
- `TT_NUMBER indica que el token es un número.`
- `TT_EOL indica que se ha leído el fin de línea.`
- `TT_EOF indica que se ha llegado al fin del flujo de entrada.`

En el código de la clase, apreciamos que se iterará hasta llegar al fin del fichero. Para cada token, se mira su tipo, y según el tipo se incrementa el contador de palabras o de números.

```
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.IOException;
import java.io.StreamTokenizer;

public class token {
    public void contarPalabrasyNumeros(String nombre_fichero) {
        StreamTokenizer sTokenizer = null;
        int contapal = 0, numNumeros = 0;
        try {
            sTokenizer = new StreamTokenizer(new FileReader(nombre_fichero));
            while (sTokenizer.nextToken() != StreamTokenizer.TT_EOF) {
                if (sTokenizer.ttype == StreamTokenizer.TT_WORD)
                    contapal++;
                else if (sTokenizer.ttype == StreamTokenizer.TT_NUMBER)
                    numNumeros++;
            }
            System.out.println("Número de palabras en el fichero: " + contapal);
            System.out.println("Número de números en el fichero: " + numNumeros);
        } catch (FileNotFoundException ex) {
            System.out.println(ex.getMessage()) ;
        } catch (IOException ex) {
            System.out.println(ex.getMessage()) ;
        }
    }

    public static void main(String[] args) {
        new Main().countWordsAndNumbers("c:\\datos.txt");
    }
}
```



Precisión Total: Ficheros de Acceso Aleatorio

RandomAccessFile

Permite saltar directamente a cualquier posición del archivo mediante punteros. Modo lectura (`"r"`) o lectura/escritura (`"rw"`).

Método seek()

Posicionamiento arbitrario de alta velocidad. Ideal para índices, tablas de datos o estructuras tipo base de datos embebida.

Lectura + Escritura

Un único flujo soporta ambas operaciones, simplificando operaciones complejas de actualización sin abrir múltiples streams.

A veces no necesitamos leer un fichero de principio a fin, sino acceder al fichero como si fuera una base de datos, donde se accede a un registro concreto del fichero. Java proporciona la clase `RandomAccessFile` para este tipo de entrada/salida.

La clase `RandomAccessFile` permite utilizar un fichero de **acceso aleatorio** en el que el programador define el formato de los registros.

```
RandomAccessFile objFile = new RandomAccessFile(ruta, modo);
```

Donde ruta es la dirección física en el sistema de archivos y modo puede ser:

- "r" para sólo lectura.
- "rw" para lectura y escritura.

La clase `RandomAccessFile` implementa los interfaces `DataInput` y `DataOutput`.

Para abrir un archivo en modo lectura haríamos:

```
RandomAccessFile in = new RandomAccessFile("input.dat", "r");
```

Para abrirlo en modo lectura y escritura:

```
RandomAccessFile inOut = new RandomAccessFile("input.dat", "rw");
```

Esta clase permite leer y escribir sobre el fichero, no se necesitan dos clases diferentes. Hay que especificar el modo de acceso al construir un objeto de esta clase: sólo lectura o lectura/escritura.

Dispone de métodos específicos de desplazamiento como `seek` y `skipBytes` para poder moverse de un registro a otro del fichero, o posicionarse directamente en una posición concreta del fichero.

No está basada en el concepto de flujos o streams.

```
boolean readBoolean();
byte readByte();

int readUnsignedByte();
short readShort();
int readUnsignedShort();
char readChar();
int readInt();
String readLine();
long readLong();
float readFloat();
double readDouble();

String readUTF();
```

```
void writeBoolean(boolean v);
void writeByte(int v);
void writeBytes(String s);
void writeShort(int v);
void writeChars(String s);
void writeChar(int v);
void writeInt(int v);
void writeLong(long v);
void writeFloat(float v);
void writeDouble(double v);

void writeUTF(String str);
```

La clase **RandomAccessFile** maneja un **puntero** que indica la posición actual el fichero. Cuando el fichero se crea, el puntero se coloca en la posición o, apuntando al principio del mismo. Las sucesivas llamadas a los métodos readXXX y writeXXX ajustan el puntero según la cantidad de bytes leídos o escritos.

Los métodos más importantes son:

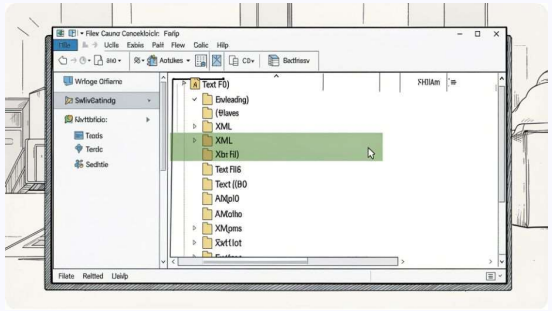
- **long getFilePointer()**: Devuelve la posición actual del puntero del fichero. Indica la posición (en bytes) donde se va a leer o escribir.
- **long length()**: Devuelve la longitud del fichero en bytes. La posición *length()* indica el final del fichero.
- **void seek (long pos)**: Coloca el puntero del fichero en una posición *pos* determinada. La posición se da como un desplazamiento en bytes desde el comienzo del fichero. La posición o indica el principio del fichero.
- **int skipBytes(int desplazamiento)**: Desplaza el puntero desde la posición actual el número de *bytes* indicados en *desplazamiento*.

Para poder determinar cuánto ocupa cada registro en un archivo de acceso aleatorio, debemos tener en cuenta los bytes de cada uno de los datos. De esta forma, dependiendo del tipo de dato que queramos almacenar, cada registro ocupará un tamaño u otro. A continuación, mostramos el tamaño en bytes que ocupa tipo de dato:

TIPO DE DATO	TAMAÑO EN BYTES
Char	2 bytes
Byte	1 byte
Short	2 bytes
Int	4 bytes
Long	8 bytes
Float	4 bytes
Double	8 bytes
Boolean	1 byte
Espacio en blanco (corresponde a un char)	2 bytes
Salto de línea (corresponde a un byte	1 byte
String	2 byte por cada char

Hay que tener en cuenta que los datos tipo String (tipo texto), en java son considerados como objetos. Por este motivo, un dato tipo String, se considera un array de caracteres tipo char. Esto quiere decir que la información de tipo String, ocupará dos bytes por cada carácter tipo char. A esta información, se le deben sumar los bytes ocupados por los espacios en blanco y los saltos de línea.

Control de Directorios: FilenameFilter



En ocasiones nos interesa ver la lista de los archivos que encajan con un determinado criterio. Así, nos puede interesar un filtro para ver los ficheros modificados después de una fecha, o los que tienen un tamaño mayor del que indiquemos, etc.

El *interface* `FilenameFilter` se puede usar para crear filtros que establezcan criterios de filtrado relativos al nombre de los ficheros. Una clase que lo implemente debe definir e implementar el método:

```
boolean accept(File dir, String nombre)
```

Este método devolverá verdadero (`true`), en el caso de que el fichero cuyo nombre se indica en el parámetro `nombre` aparezca en la lista de los ficheros del directorio indicado por el parámetro `dir`.

En el siguiente ejemplo vemos cómo se listan los ficheros de la carpeta c:\datos que tengan la extensión .odt. Usamos `try` y `catch` para capturar las posibles excepciones, como que no exista dicha carpeta.

```
import java.io.File;
import java.io.FilenameFilter;
public class Filtrar implements FilenameFilter {
   String extension;
   Filtrar (String extension){ //constructor
      this.extension=extension;
   }
   public boolean accept(File dir, String name){
      return name.endsWith(extension);
   }
   public static void main(String[] args) {
      try {
         File fichero=new File("c:\datos\.");
         String[] listadeArchivos = fichero.list();
         listadeArchivos = fichero.list(new Filtrar(".odt"));
         int numarchivos = listadeArchivos.length;
         if (numarchivos < 1)
            System.out.println("No hay archivos que listar");
         else {
            for(int conta = 0; conta < listadeArchivos.length; conta++)
               System.out.println(listadeArchivos[conta]);
         }
      } catch (Exception ex) {
         System.out.println("Error al buscar en la ruta indicada");
      }
   }
}
```

EJERCICIOS

20. Copiar el contenido de un fichero en otro.

21. Contar números y palabras de un fichero de texto.

22. Inserta en un fichero los datos de dos arrays, uno con el nombre de una serie de personas y otro con sus edades.

23. Visualiza los datos guardados en el ejercicio anterior en el mismo orden que se escribieron, es decir, primero el nombre y luego la edad.

24. Insertar el apellido, departamento y salario de empleados en un fichero aleatorio. Se obtienen de arrays y se insertan de forma secuencial, por lo que no será necesario usar el método seek(). Por cada empleado se insertará también un identificador, que comenzará en 1. La longitud del registro de cada empleado es 36 bytes:

- id (entero): 4 bytes
- apellido (10 caracteres x 2 bytes): 20 bytes
- departamento (entero): 4 bytes
- salario (double): 8 bytes

25. Visualizar el fichero del ejercicio anterior. La posición inicial será 0 y para recuperar los siguientes registros hay que sumar 36 a la variable utilizada para el posicionamiento.

26. Consultar los datos del empleado con identificador 5. No es necesario recorrer todos los registros anteriores, para acceder a su posición, tener en cuenta que cada empleado ocupa 36 bytes.

27. Añadir un nuevo empleado al final.

28. Listar los ficheros de una carpeta que tengan la extensión .java

Serialización y Deserialización



Hasta ahora hemos visto cómo leer y escribir datos en un fichero, tanto cadenas de caracteres como tipos primitivos Java. Ahora veremos cómo intercambiar otros tipos de objetos Java entre una aplicación y el disco.

La serialización convierte objetos complejos en secuencias de bytes almacenables; la deserialización reconstruye la jerarquía de memoria desde un archivo.

Serializable

Interfaz marcador que habilita la conversión de objetos a bytes. El runtime gestiona automáticamente la jerarquía de clases.

ObjectInputStream / ObjectOutputStream

Mecanismos de persistencia y recuperación del estado completo de un objeto, incluyendo sus referencias internas.

¿Qué es la serialización? Es un proceso por el que un objeto se convierte en una secuencia de bytes con la que más tarde se podrá reconstruir el valor de sus variables. Esto permite guardar un objeto en un archivo.

Todos los tipos primitivos en Java son serializables por defecto. (Al igual que los arrays y otros muchos tipos estándar). Para leer y escribir objetos serializables a un stream se utilizan las clases java: **ObjectInputStream** y **ObjectOutputStream**.

Serialización de objetos

Para que un objeto pueda ser almacenado en disco, es necesario que la clase a la que pertenece sea serializable. Esta característica la poseen todas aquellas clases que implementen la interfaz **java.io.Serializable**.

La interfaz Serializable no contiene ningún método, basta con que una clase la implemente para que sus objetos puedan ser serializados por la máquina virtual y por tanto almacenados en disco.

El siguiente listado corresponde a una clase Persona de tipo JavaBean, cuyos objetos encapsulan el nombre y la edad de una persona. Estos objetos podrán ser transferidos a disco pues Persona implementa la interfaz Serializable:

```
import java.io.Serializable;
public class Persona implements Serializable {
   private String name;
   private int edad;
   public Persona (String n, int e) {
      name = n;
      edad = e;
   }
   public void setName {String n) {
      name=n;
   }
   public String getName(){
      return name;
   }
   public void setEdad(int e) {
      edad=e ;
   }
   public int getEdad(){
      return edad;
   }
}
```

Creación de un objeto ObjectOutputStream

La escritura de objetos en disco se realiza a través de **ObjectOutputStream**. Para crear un objeto de esta clase es necesario disponer del **FileOutputStream** asociado al fichero donde se va a realizar la escritura.

```
FileOutputStream fisal = new FileOutputStream(“cadenas.out”);
ObjectOutputStream oos = new ObjectOutputStream(fisal);
oos.writeObject();
```

Una vez creado el objeto, la clase dispone del método ***writeObject(Object obj)*** para realizar la escritura del objeto en el disco. El siguiente programa almacena en un fichero de disco un objeto de la clase Persona definida anteriormente:

```
import java.io.*;
public class Almacenamiento {
   public static void main(String[] args) throws IOException{
      FileOutputStream fs= new FileOutputStream("datos.obj");
      ObjectOutputStream os=new ObjectOutputStream(fs);
      os.writeObject (new Persona ("paco", 40));
      os.close(); //cierra el stream
   }
}
```

Lectura de objetos desde un fichero

Por último, para leer objetos de un fichero que han sido almacenados mediante **ObjectOutputStream**, se deberá utilizar un objeto **ObjectInputStream**.

Creación de un objeto ObjectInputStream

La clase **ObjectInputStream** dispone del método Object ***readObject()*** para devolver los objetos almacenados en el fichero. El constructor de esta clase requiere un objeto **FileInputStream** asociado al fichero de donde se leerá el objeto. El siguiente programa recupera el objeto Persona almacenado en el fichero datos.obj por la clase del ejemplo anterior y muestra sus datos en pantalla:

```
import java.io.*;
public class Recuperacion {
   public static void main(String[] args) throws IOException, ClassNotFoundException {
      FileInputStream fs=new FileInputStream ("Datos.obj");
      ObjectInputStream os=new ObjectInputStream(fs);
      //se debe realizar un casting al tipo original
      Persona p=(Persona) os.readObject();
      System.out.println (p.getName());
      System.out.println (p.getEdad());
      os.close();
   }
}
```

Tras la ejecución del programa anterior se visualizará en pantalla:

paco

40

El método ***readObject()***, además de **IOException**, declara también una excepción de tipo **ClassNotFoundException** que habrá que capturar.

Deserialización de objetos

Cuando se recupera un objeto del disco mediante la llamada a ***readObject()***, se produce la deserialización del objeto que básicamente consiste en la reconstrucción de éste a partir de la información recuperada.

Durante este proceso, los datos miembro no serializables (aquéllos que han sido heredados de una clase no serializable) serán inicializados utilizando el constructor por defecto de su clase. Por el contrario, los datos miembro de la clase del objeto señalizado serán restaurados con los valores almacenados

Problema con los ficheros de objetos

Al crear un fichero de objetos se crea una cabecera inicial con información, y a continuación se añaden los objetos. Si el fichero se utiliza de nuevo para añadir más registros, se crea una nueva cabecera y se añaden objetos a partir de esa cabecera. El problema surge al leer el fichero cuando en la lectura se encuentra con la segunda cabecera, y aparece la excepción ***StreamCorruptedException***, y no podemos leer más objetos.

La cabecera se crea cada vez que se pone ***new ObjectOutputStream(fichero)***. Para que no se añadan estas cabeceras se redefine la clase ***ObjectOutputStream*** creando una nueva clase que la herede, y dentro de esa clase se redefine el método ***writeStreamHeader()***, que es el que escribe las cabeceras, para que no haga nada.

```
public class MiObjectOutputStream extends ObjectOutputStream {
   /** Constructor que recibe OutputStream */
   public MiObjectOutputStream(OutputStream out) throws IOException{
      super(out);
   }
   /** Constructor sin parametros */
   protected MiObjectOutputStream() throws IOException, SecurityException{
      super();
   }
   /* Redefinicion del método de escribir la cabecera para que no haga nada. */
   protected void writeStreamHeader() throws IOException{
   }
}
```

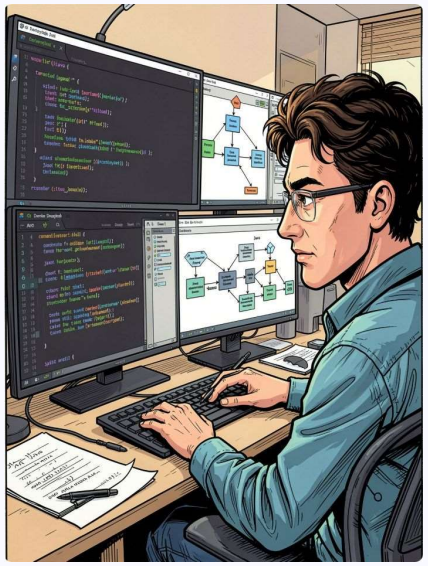
Y dentro de nuestro programa, a la hora de abrir el fichero para añadir nuevos objetos, se pregunta si ya existe. En caso de que sí exista, se crea un objeto de la clase redefinida, y si no existe, el fichero se crea con la clase *ObjectOutputStream*.

```
File fichero = new File(nombrefichero);
ObjectOutputStream dataOS;
if (!fichero.exist()){
   FileOutputStream fileout;
   fileout = new FileOutputStream(fichero);
   dataOS = new ObjetOutputStream(fileout);
} else {
   dataOS = new MiObjectOutputStream (new FileOutputStream(fichero,true));
}
```

EJERCICIOS

29. Escribe objetos Persona en un fichero. Puedes tener los nombres y edades almacenados en arrays o bien solicitarlos por teclado.

30. Lee y muestra por pantalla los datos de las personas almacenadas en el fichero anterior.



Datos Estructurados: Ficheros XML

XML es un lenguaje de marcas muy estandarizado utilizado para almacenar e intercambiar datos de forma legible, caracterizado por estructurarse a través de etiquetas.

En java, podemos crear nuestros propios XML para luego tratarlos. Los ficheros XML se pueden utilizar para proporcionar datos a una base de datos, o para almacenar copias de partes de contenidos de la base de datos. También se utilizan para escribir ficheros de configuración de programas o en el protocolo SOAP (Simple Object Access Protocol), para ejecutar comandos en servidores remotos...

XML consigue estructurar cualquier tipo de información jerárquica y es un lenguaje utilizado ampliamente en el desarrollo de la World Wide Web que cuenta con herramientas y estándares de programación para leer documentos XML. Estas herramientas o programas que leen el lenguaje XML y comprueban si el documento es válido sintácticamente, se denominan analizadores o "parsers".

Un parser XML es un módulo, biblioteca o programa que se ocupa de analizar, clasificar y transformar un archivo de XML en una representación interna, extrayendo la información contenida en cada una de las etiquetas y relacionándola de acuerdo con su posición en la jerarquía. Estos analizadores o parsers pueden ser secuenciales como **SAX, STAX** o jerárquicos como DOM.

Analizadores secuenciales

Los analizadores secuenciales, que permiten extraer el contenido a medida que se van descubriendo las etiquetas de apertura y cierre, se denominan analizadores sintácticos. Son analizadores muy rápidos, pero presentan el problema de que cada vez que se necesita acceder a una parte del contenido necesario releer todo el documento de arriba a abajo.

En Java hay dos analizadores secuenciales: **SAX**, que es el acrónimo de **Simple API for XML**. Es un analizador muy usado en varias bibliotecas de tratamiento de datos XML, pero no suele usarse en aplicaciones finales y **STAX**, Streaming API for XML, posterior a SAX y que lo ha superado.

Analizadores jerárquicos

Generalmente, las aplicaciones finales que necesitan trabajar con datos XML suelen usar analizadores jerárquicos, porque además de realizar un análisis secuencial que les permite clasificar el contenido, se almacenan en la memoria RAM siguiendo la estructura jerárquica detectada en el documento. Esto facilita mucho las consultas que haya que repetir varias veces, dado que las estructuras jerárquicas de la memoria RAM tienen un rendimiento de acceso parcial a los datos muy eficiente.

Los analizadores jerárquicos guardan todos los datos del XML en memoria dentro de una estructura jerárquica. Son ideales para aplicaciones que requieran una consulta continua de los datos. El formato de la estructura donde se almacena la información en la memoria RAM ha sido especificado por el organismo internacional W3C (World Wide Web Consortium) y se suele conocer como **DOM (Document Object Model)**. Es una estructura que HTML y javascript han popularizado mucho y se trata de una especificación que Java materializa en forma de interfaces. La principal se denomina Document y representa todo un documento XML. Al tratarse de una interfaz, puede ser implementada por varias clases.

DOM — Document Object Model

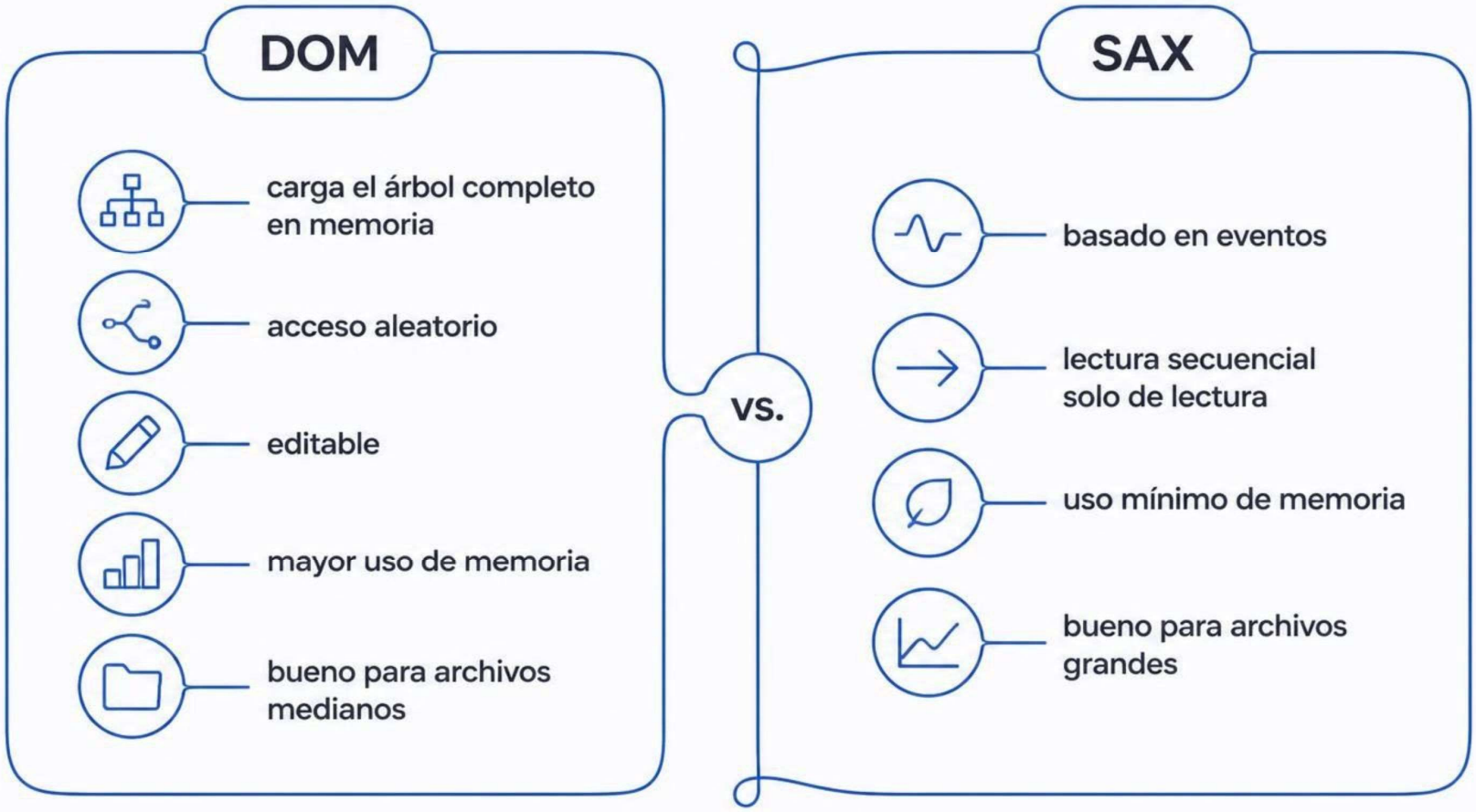
Carga el **árbol completo en memoria**. Permite navegación aleatoria, modificación y consulta XPath. Ideal para ficheros medianos con manipulación intensa.

- Acceso aleatorio a cualquier nodo
- Mayor consumo de memoria

SAX — Simple API for XML

Basado en **eventos**: el parser notifica al programa conforme lee. Extremadamente eficiente para archivos gigantescos donde la memoria es cuello de botella.

- Solo lectura, flujo secuencial
- Consumo mínimo de memoria


DOM
carga el árbol completo en memoria
acceso aleatorio
editable
mayor uso de memoria
bueno para archivos medianos
vs.
SAX
basado en eventos
lectura secuencial solo de lectura
uso mínimo de memoria
bueno para archivos grandes

Comparativa analizadores

CARACTERÍSTICA	STAX	SAX	DOM	TRAX
Tipo de API	Pull, streaming	Push, streaming	En memoria	Regla XSLT
Facilidad de uso	Alta	Media	Alta	Media
Capacidad XPath	No	No	Si	Si
Eficiencia de CPU y memoria	Buena	Buena	Varia	Varia
Solo hacia adelante	Si	Si	No	No
Lee XML	Si	Si	Si	Si
Escribe XML	Si	No	Si	Si
Crear, Leer, Modificar, Borrar	No	No	Si	No

La escritura de la información contenida en el DOM se puede secuenciar en forma de texto utilizando una utilidad de Java llamada **Transformer**. Se trata de una utilidad que permite realizar fácilmente conversiones entre diferentes representaciones de información jerárquica. Es capaz, por ejemplo, de pasar la información contenida en un objeto Documento a un archivo de texto en formato XML. También sería capaz de hacer la operación inversa, pero el mismo DocumentBuilder ya se encarga de ello. Transformer es también una clase abstracta y requiere de una *fábrica* para poder ser instanciada. La clase **Transformer** puede trabajar con multitud de contenedores de información porque en realidad trabaja con un par de tipos adaptadores (clases que hacen compatibles jerarquías diferentes) que se llaman **Source** y **Result**. Las clases que implementen estas interfaces se encargarán de hacer compatible un tipo de contenedor específico al requerimiento de la clase Transformer. Así, disponemos de las clases DOMSource, SAXSource o StreamSorce como adaptadores del contenedor de la fuente de información (DOM, SAX o Stream respectivamente). DOMResult, SAXResult o StreamResult son los adaptadores equivalentes del contenedor destino.

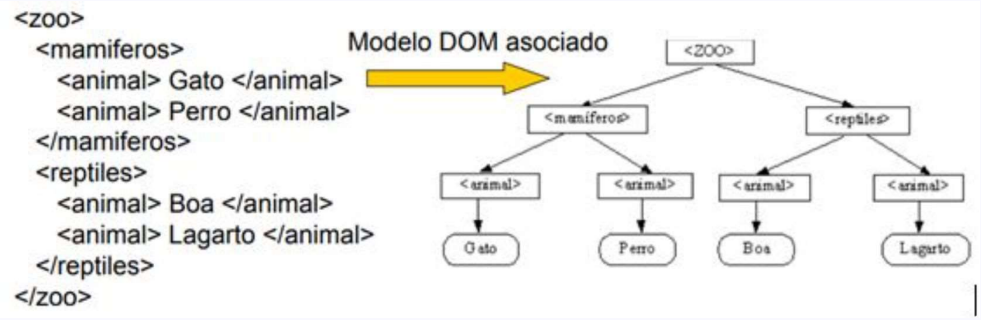
El código básico para realizar una transformación de DOM a archivo XML sería el siguiente:

```
// Creación de una instancia Transformer
Transformer trans = TransformerFactory.newInstance( ).newTransformer();

//Creación de los adaptadores Source y Results a partir de un Documento y un File.
StreamResult result = new StreamResult (file);
DOMSource source = new DOMSource (doc);
trans.transform (source, result);
```

La estructura DOM

La estructura DOM toma la forma de un árbol, donde cada parte del XML se encontrará representada en forma de nodo. **En función de la posición en el documento XML, hablaremos de diferentes tipos de nodos. El nodo principal que representa todo el XML entero se denomina documento**, y las diversas **etiquetas**, incluida la etiqueta raíz, se conocen como **nodos elemento.** El **contenido textual** de una etiqueta se instancia como **nodo de tipo TextElement** y los **atributos** como nodos de **tipo Atribute.** Cada nodo específico dispone de métodos para acceder a sus datos concretos (nombre, valor, nodos hijos, nodo padre, etc.).



El DOM resultante obtenido de un XML termina siendo una copia exacta del archivo, pero con una disposición distinta. Tanto el DOM como el XML tendrán información no visible, como los retornos de carro, que deben tenerse en cuenta para saber cómo procesar correctamente el contenido y evitar sorpresas un poco comprensibles.

Debes tener en cuenta que al mapear XML los retornos de carro se interpretan en DOM como un *hijo* y que el contenido textual de las etiquetas se plasma en el DOM como un nodo hijo de la etiqueta contenedora. Es decir, para obtener el texto de una etiqueta hay que obtener el primer hijo de ésta.

La interfaz Document contempla un conjunto de métodos para seleccionar diferentes partes del árbol a partir del nombre de la etiqueta o de un atributo identificador. Las partes del árbol se devuelven como objetos Element, los cuales representan un nodo y todos sus hijos. De este modo, podremos ir explorando partes del árbol sin necesidad de tener que pasar por todos los nodo

DOM define varias **interfaces**, las más comunes son las siguientes:

- **Document** – Representa al documento XML completo. Generalmente nos referiremos a él como árbol DOM. Proporciona información del documento. Permite crear nuevos nodos en el documento.
- **Element** – expone propiedades y métodos para manipular los elementos del documento y sus atributos.
- **Node** – Representa a cualquier nodo del documento.
- **NodeList** – Colección de nodos a los que se puede acceder por medio de un índice.
- **Attr** – Representa un atributo de un elemento.
- **Text** – Contenido de un elemento o atributo.
- **CharacterData** – Representa a los datos carácter presentes en el documento. Proporciona atributos y métodos para manipular los datos de caracteres.
- **DocumentType** - Proporciona información contenida en la etiqueta <!DOCTYPE>.

Los **métodos DOM más usuales** son:

- **document.getDocumentElement()** – Retorna el elemento raíz del documento.
- **document.getElementById('id_del_elemento')** - permite seleccionar un elemento del documento por medio del valor del atributo id.
- **document.getElementsByTagName('elemento')** - obtenemos todos los elementos de un tipo
- **document.getElementsByTagName('elemento')[índice_del_elemento]** - elegimos uno de ellos en concreto
- **Node.getNodeType()** - Obtiene el tipo del nodo.
- **Node.getNodeName()** - Obtiene el nombre del nodo.
- **Node.getNodeValue()** - Obtiene el valor del nodo.
- **Node.getTextContent()** - Obtiene el contenido de texto del nodo.
- **Node.getChildNodes ()** - Obtiene todos los nodos secundarios del nodo.
- **Node.getFirstChild()** – Retorna el primer nodo hijo del nodo.
- **Node.getLastChild()** – Retorna el ultimo nodo hijo del nodo.
- **Node.getNextSibling()** – Retorna el siguiente hermano de un nodo.
- **Node.getPreviousSibling()** – Retorna el hermano anterior de un nodo.
- **Node.getAttribute(attrName)** – Retorna el atributo del nodo con el nombre que se pasa como atributo.
- **NodeList.getLength()** - obtiene la longitud de la colección
- **item()** - Obtiene el nodo al que apunta el valor del índice de acuerdo con el valor del índice entrante

Los objetos Elemento disponen de métodos para añadir nuevos hijos (**appendChild**) o asignar el valor a un atributo (**setAttribute**). También permiten la consulta del valor de los atributos (**getAttribute**) o la navegación por los nodos del árbol (**getParentNode**, para obtener el padre; **getFirstChild/getLastChild**, para obtener el primer / último hijo, o **getNextSibling** para navegar de hermano en hermano).

El objeto **Document** hará de factory para cualquier nodo del documento. La creación de nuevos elementos (etiquetas) implicará el uso de **createElement**. Si queremos crear contenido textual habrá llamar al método **createTextNode**, y si lo que queremos son comentarios llamaremos **createComment**.

La creación de nodos no implica la ubicación de este dentro del árbol. Es decir, además de crearlos habrá asignarlos a un padre usando **appendChild**.

Para crear un atributo tenemos dos opciones:

```
Attr attr = doc.createAttribute("nombre_atributo"); //crear el atributo

attr.setValue("valor_atributo"); //dar valor al atributo

elemento.setAttributeNode(attr); //asignar el atributo al elemento
```

O bien añadir el atributo de una vez:

```
elemento.setAttribute("nombre_atributo","valor_atributo");
```

Ejemplo Completo DOM

A continuación, vamos a crear un fichero XML a partir del fichero aleatorio de empleados creado antes:

```
//Lo primero es importar los paquetes necesarios:
import org.w3c.dom.*;
import javax.xml.parsers.*;
import javax.xml.transform.*;
import javax.xml.transform.dom.*;
import javax.xml.transform.stream.*;
import java.io.*;
public class CrearEmpleadoXml {
   public static void main(String args[]) throws IOException{
      File fichero = new File("AleatorioEmple.dat");
      RandomAccessFile file = new RandomAccessFile(fichero, "r");
      int id, dep, posicion=0; //para situarnos al principio del fichero
      Double salario;
      char apellido[] = new char[10], aux;
```

```
//Crear una instancia de DocumentBuilderFactory para construir el parser.
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
//Encerrar en un bloque try - catch por si se produce la excepción ParserConfigurationException
try{
   DocumentBuilder builder = factory.newDocumentBuilder();
     //Crear un documento vacío de nombre document, con el nodo raíz de nombre Empleados y
     //asignamos la versión del XML. La interfaz DOMImplementation permite crear objetos Document con nodo raíz
   DOMImplementation implementation = builder.getDOMImplementation();
   Document document = implementation.createDocument(null, "Empleados", null);
   document.setXmlVersion("1.0");
   //recorrer el fichero con los datos de los empleados y por cada registro crear un nodo
   //empleado con 4 hijos (id, apellido, departamento y salario)
   for(;;) {
      file.seek(posicion); //nos posicionamos
      id=file.readInt(); //obtenemos id de empleado
      for (int i = 0; i < apellido.length; i++) {
         aux = file.readChar();
         apellido[i] = aux;
      }
      String apellidos = new String(apellido);
      dep = file.readInt();
      salario = file.readDouble();
      if(id>0) { //id validos a partir de 1
         Element emple = document.createElement("empleado"); //crear el nodo empleado
         document.getDocumentElement().appendChild(emple); //lo pegamos en la raíz del documento
          //Añadir los hijos de ese nodo (raíz). Se añaden en el método CrearElemento()
         //El método recibe el nombre del nodo hijo (id, apellido, dep o salario) y sus textos o valores en
         //formato String (1, FERNANDEZ, 10, 1000.45), el nodo al que se va a añadir (raíz) y el documento (document)
          CrearElemento("id",Integer.toString(id), emple, document); //añadir ID
          CrearElemento("apellido",apellidos.trim(), emple, document); //Apellido
          CrearElemento("dep",Integer.toString(dep), emple, document); //añadir DEP
          CrearElemento("salario",Double.toString(salario), emple, document); //añadir salario
      }
      posicion= posicion + 36; // me posiciono para el sig empleado
      if (file.getFilePointer() == file.length()) break;
   }//fin del for que recorre el fichero
```

```
        //Se crea la fuente XML a partir del documento
        Source source = new DOMSource(document);
        //Se crea el resultado en el fichero Empleados.xml
        Result result = new StreamResult(new java.io.File("Empleados.xml"));
        //Se obtiene ahora un TransformerFactory
        Transformer transformer = TransformerFactory.newInstance().newTransformer();
        //se realiza la transformación
        transformer.transform(source, result);
      } catch(Exception e){
        System.err.println("Error: "+e);
      }
      file.close(); //cerrar fichero
    }//fin de main
    //Inserción de los datos del empleado
    static void CrearElemento(String datoEmple, String valor, Element emple, Document document){
      //Para crear el nodo hijo (<id> o <apellido> o <dep> o <salario>) se escribe:
      Element elem = document.createElement(datoEmple);
      //Para añadir su valor o su texto se usa el método createTextNode(String)
      Text text = document.createTextNode(valor); //damos valor
      //A continuación se añade el nodo hijo a la raíz (empleado) y su valor al nodo hijo
      emple.appendChild(elem); //pegamos el elemento hijo a la raíz
      elem.appendChild(text); //pegamos el valor
    }
}//fin de la clase
/*Al final se generaría algo similar a esto por cada empleado
<empleado><id>1</id><apellido>FERNANDEZ</apellido><dep>10</dep><salario>1000.45</salario></empleado>
*/
```

Para **leer** un documento XML:

```
import java.io.File;
import javax.xml.parsers.*;
import org.w3c.dom.*;
public class LecturaEmpleadoXml {
   public static void main(String[] args) {
      //creamos una instancia de DocumentBuilderFactory para construir el parser y cargamos el documento con el método parse()
     DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
     try {
       DocumentBuilder builder = factory.newDocumentBuilder();
       Document document = builder.parse(new File("Empleados.xml"));
       //Elimina nodos vacios y combina adyacentes en caso de que los hubiera
       document.getDocumentElement().normalize();
       System.out.printf("Elemento raiz: %s %n",
       document.getDocumentElement().getNodeName())
       //crea una lista con todos los nodos empleado
       NodeList empleados = document.getElementsByTagName("empleado");
       System.out.printf("Nodos empleado a recorrer: %d %n",
       empleados.getLength());
       //recorrer la lista de nodos. Por cada nodo se obtienen sus etiquetas y sus valores
       for (int i = 0; i < empleados.getLength(); i ++) {
         //obtener un nodo empleado
         Node emple = empleados.item(i);
         if (emple.getNodeType() == Node.ELEMENT_NODE) {//tipo de nodo
           //obtener los elementos del nodo
           Element elemento = (Element) emple;
           System.out.printf("ID = %s %n", elemento.getElementsByTagName("id").item(0).getTextContent());
               System.out.printf(" * Apellido = %s %n", elemento.getElementsByTagName("apellido").item(0).getTextContent());
               System.out.printf(" * Departamento = %s %n",elemento.getElementsByTagName("dep").item(0).getTextContent());
               System.out.printf(" * Salario = %s %n", elemento.getElementsByTagName("salario").item(0).getTextContent());
         }
       }
     } catch (Exception e) {
       e.printStackTrace();
     }
   }//fin de main
}//fin de la clase
```

SAX

SAX (API Simple para XML) es un conjunto de clases e interfaces que ofrecen una herramienta muy útil para el procesamiento de documentos XML. Permite analizar los documentos de forma secuencial (es decir, no se carga todo el fichero en memoria como hace DOM), esto implica poco consumo de memoria, aunque los documentos sean de gran tamaño, en contraposición, impide tener una visión global del documento que se va a analizar.

SAX es más complejo de programar que DOM, es un API totalmente escrita en Java e incluida dentro del JRE que nos permite crear nuestro propio parser de XML.

La lectura de un documento XML produce eventos que ocasiona la llamada a métodos, los eventos son encontrar la etiqueta de inicio (*startDocument*) y fin (*endDocument*) del documento, la etiqueta de inicio (*startElement*) y fin (*endElement*) de un elemento, los caracteres entre etiquetas (*characters()*)...

Documento XML (alumnos.xml)	Métodos asociados a eventos del documento
<?xml version="1.0"?> <listadealumnos> <alumno> <nombre> Juan </nombre> <edad> 19 </edad> </alumno> <alumno> <nombre> Maria </nombre> <edad> 20 </edad> </alumno> </listadealumnos>	startDocument() startElement() startElement() startElement() characters() endElement() startElement() characters() endElement() endElement() startElement() startElement() characters() endElement() startElement() characters() endElement() endElement() endElement() endDocument()

Los objetos que tratarán los eventos normalmente serán implementaciones de las siguientes interfaces:

- **ContentHandler**: recibe las notificaciones de los eventos que ocurren en el documento.

- **DTDHandler**: recoge eventos relacionados con la DTD.

- **ErrorHandler**: define métodos de tratamiento de errores.

- **EntityResolver**: sus métodos se llaman cada vez que se encuentra una referencia a una entidad.

- **DefaultHandler**: clase que provee una implementación por defecto para todos sus métodos. El programador definirá los métodos que sean utilizados por el programa. Esta clase es la que extendemos para poder crear nuestro *parser* de XML. En el ejemplo siguiente, la clase se llama *GestionContenido* y se tratan solo los eventos básicos:

- ***startDocument:*** se produce al comenzar el procesado del documento XML.
- ***endDocument:*** se produce al finalizar el procesado del documento XML.
- ***startElement:*** se produce al comenzar el procesado de una etiqueta XML. Es aquí donde se leen los atributos de las etiquetas.
- ***endElement:*** se produce al finalizar el procesado de una etiqueta XML.
- ***Characters:*** se produce al encontrar una etiqueta de texto.

Para indicar al procesador XML los objetos que realizarán el tratamiento se utiliza alguno de los siguientes métodos incluidos dentro del objeto *XMLReader*: *setContentHandler(), setDTDHandler(), setEntityResolver() y setErrorHandler()*; cada uno trata un tipo de evento y está asociado con una interfaz determinada. En el siguiente ejemplo se utilizará *setContentHandler()* para tratar los eventos del documento.

Ejemplo:

```
//Incluir las clases e interfaces SAX
import java.io.*;
import org.xml.sax.Attributes;
import org.xml.sax.InputSource;
import org.xml.sax.SAXException;
import org.xml.sax.XMLReader;
import org.xml.sax.helpers.DefaultHandler;
import org.xml.sax.helpers.XMLReaderFactory;
public class PruebaSax1 {
   public static void main(String[] args) throws FileNotFoundException, IOException, SAXException{
      //Se crea un objeto procesador de XML, un XMLReader. Durante su creación se puede producir
      //una excepción SAXException (se incluye en el método main)
      XMLReader procesadorXML = XMLReaderFactory.createXMLReader();
      //setContentHandler() para tratar los eventos del documento
      GestionContenido gestor= new GestionContenido();
      procesadorXML.setContentHandler(gestor);
      //define el fichero XML que se va a leer mediante un objeto InputSource
      InputSource fileXML = new InputSource("alumnos.xml");
      //procesa el documento XML mediante el método parse() del objeto XMLReader.
      //Le pasamos un objeto InputSource
      procesadorXML.parse(fileXML);
   }
}//fin PruebaSax1
```

```
class GestionContenido extends DefaultHandler {
   public GestionContenido() {
      super();
   }
   public void startDocument() {
      System.out.println("Comienzo del Documento XML");
   }
   public void endDocument() {
      System.out.println("Final del Documento XML");
   }
   public void startElement(String uri, String nombre, String nombreC, Attributes atts) {
      System.out.printf("\t Principio Elemento: %s %n",nombre);
      if (atts.getLength() > 0){
         for (int i=0; i<atts.getLength(); i++){
            System.out.print("\tAtributo: " + atts.getLocalName(i) + " ");
            System.out.println(atts.getValue(i));
         }
      }
   }
   public void endElement(String uri, String nombre, String nombreC) {
      System.out.printf("\tFin Elemento: %s %n", nombre);
   }
   public void characters(char[] ch, int inicio, int longitud) throws SAXException {
      String car=new String(ch, inicio, longitud);
      //quitar saltos de línea
      car = car.trim();
      if (car.length() > 0)
         System.out.println ("\tCaracteres: " + car);
   }
} //fin GestionContenido
```

El resultado de ejecutar el programa sobre el fichero alumnos.xml sería:

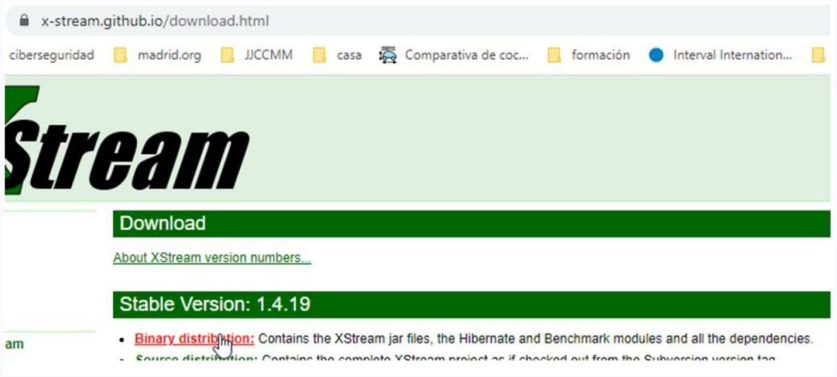
```
Comienzo del Documento XML
Principio Elemento: listadealumnos
Principio Elemento: alumno
Principio Elemento: nombre
Caracteres: Juan
Fin Elemento: nombre
Principio Elemento: edad
Caracteres: 19
Fin Elemento: edad
Fin Elemento: alumno
Principio Elemento: alumno
Principio Elemento: nombre
Caracteres: Maria
Fin Elemento: nombre
Principio Elemento: edad
Caracteres: 20
Fin Elemento: edad
Fin Elemento: alumno
Fin Elemento: listadealumnos
Final del Documento XML
```

Serialización de objetos a XML

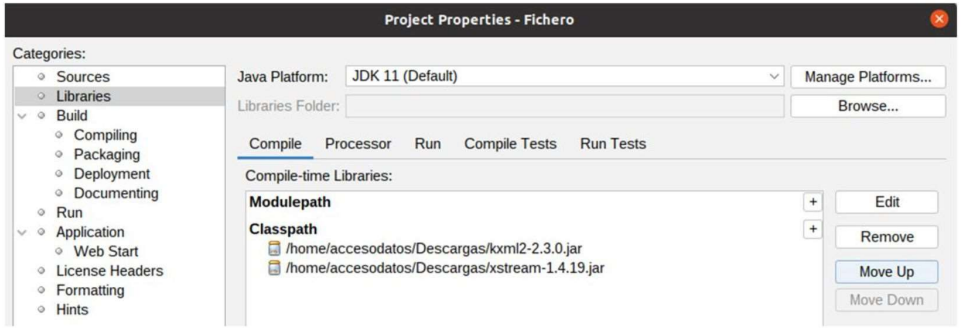
Para conseguirlo se utiliza la librería XStream**, que descargamos del sitio**

***http://x-stream.github.io/download.html*. Para el ejemplo se ha descargado el fichero**

*xstream-distribution-1.4.19-bin.zip***, que hemos de descomprimir y buscar el JAR** *xstream-1.4.19.jar*., de la carpeta ***lib***. También necesitamos el fichero *kxml2-2.3.0.jar* de la carpeta ***lib/stream***. Es necesario añadir estos dos ficheros a nuestro proyecto:



propiedades del proyecto / bibliotecas / añadir JAR



Partimos del fichero *FichPersona.dat*, que contiene objetos *Persona*. Creamos una lista de objetos persona y la convertimos en un fichero de datos XML. Necesitaremos la clase *Persona* (ya definida) y la clase *ListaPersonas*, en la que se define una lista de objetos *Persona* que pasaremos al fichero XML:

Persona.java

```
import java.io.Serializable;
public class Persona implements Serializable{
   private String nombre;
   private int edad;
   public Persona(String nombre,int edad) {
      this.nombre=nombre;
      this.edad=edad;
   }
   public Persona() {
      this.nombre=null;
   }
   public void setNombre(String nom){nombre=nom;}
   public void setEdad(int ed){edad=ed;}
   public String getNombre(){return nombre;}
   public int getEdad(){return edad;}
}//fin Persona
```

ListaPersonas.java

```
import java.util.ArrayList;
import java.util.List;
public class ListaPersonas {
    private List<Persona> lista = new ArrayList<Persona>();
    public ListaPersonas(){
    }
    public void add(Persona per) {
        lista.add(per);
    }
    public List<Persona> getListaPersonas() {
        return lista;
    }
}
```

El proceso consistirá en recorrer el fichero FichPersona.dat para crear una lista de personas que después se insertarán en el fichero Personas.xml:

```
import java.io.*;
import com.thoughtworks.xstream.XStream;
public class EscribirPersonas {
   public static void main(String[] args) throws IOException, ClassNotFoundException {
      File fichero = new File("FichPersona.dat");
      FileInputStream filein = new FileInputStream(fichero); //flujo de entrada
      //conecta el flujo de bytes al flujo de datos
      ObjectInputStream dataIS = new ObjectInputStream(filein);
      System.out.println ("Comienza el proceso de creación del fichero a XML ...");
      //Creamos un objeto Lista de Personas
      ListaPersonas listaper = new ListaPersonas();
      try {
         while (true) { //lectura del fichero
            //leer una Persona
            Persona persona= (Persona) dataIS.readObject();
            listaper.add(persona); //añadir persona a la lista
         }
      } catch (EOFException eo) {}
      dataIS.close(); //cerrar stream de entrada
```

```
        try {
            //Crear una instancia de la clase Stream
            XStream xstream = new XStream();
            //cambiar de nombre a las etiquetas XML con un alias para las clases
            xstream.alias("ListaPersonasMunicipio", ListaPersonas.class);
            xstream.alias("DatosPersona", Persona.class);
            //también pueden crearse alias para los campos
            xstream.aliasField("NombreAlumno", Persona.class, "nombre");
            xstream.aliasField("EdadAlumno", Persona.class, "edad");
            //quitar etiqueta lista (atributo de la claseListaPersonas)
            xstream.addImplicitCollection (ListaPersonas.class, "lista");
            //Insertar los objetos en el XML
            xstream.toXML(listaper, new FileOutputStream("Personas.xml"));
            System.out.println("Creado fichero XML....");
        }catch (Exception e) {e.printStackTrace();}
    } // fin main
} //fin EscribirPersonas
```

El fichero generado es el siguiente:

```
<ListaPersonasMunicipio>
<DatosPersona>
   <nombre>Ana</nombre>
   <edad>14</edad>
</DatosPersona>
<DatosPersona>
   <nombre>Luis Miguel</nombre>
   <edad>15</edad>
</DatosPersona>
<DatosPersona>
   <nombre>Alicia</nombre>
   <edad>13</edad>
</DatosPersona>
<DatosPersona>
   <nombre>Pedro</nombre>
   <edad>15</edad>
</DatosPersona>
<DatosPersona>
   <nombre>Manuel</nombre>
   <edad>16</edad>
</DatosPersona>
<DatosPersona>
   <nombre>Andrés</nombre>
   <edad>12</edad>
</DatosPersona>
<DatosPersona>
   <nombre>Julio</nombre>
   <edad>16</edad>
</DatosPersona>
<DatosPersona>
   <nombre>Antonio</nombre>
   <edad>14</edad>
</DatosPersona>
<DatosPersona>
   <nombre>María Jesús</nombre>
   <edad>13</edad>
</DatosPersona>
</ListaPersonasMunicipio>
```

Para leer el XML se deben utilizar los métodos *alias()* y *addImplicitCollection()*, que ya se usaron en la escritura del mismo.

Para obtener el objeto con la lista de personas se utiliza el método ***fromXML(InputStream input)***, que devuelve un objeto tipo Object.

```
import java.io.*;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import com.thoughtworks.xstream.XStream;
import com.thoughtworks.xstream.security.AnyTypePermission;
public class LeerPersonas {
   public static void main(String[] args) throws IOException {
      //Crear una instancia de la clase Stream
      XStream xstream = new XStream();
      xstream.addPermission(AnyTypePermission.ANY);
      //cambiar de nombre a las etiquetas XML con un alias para las clases
      xstream.alias("ListaPersonasMunicipio", ListaPersonas.class);
      xstream.alias("DatosPersona", Persona.class);
      xstream.aliasField("NombreAlumno", Persona.class, "nombre");
      xstream.aliasField("EdadAlumno", Persona.class, "edad");
      xstream.addImplicitCollection(ListaPersonas.class, "lista");
```

```
        //obtener el objeto con la lista de personas
        ListaPersonas listadoTodas = (ListaPersonas)
        xstream.fromXML(new FileInputStream("Personas.xml"));
        System.out.println("Numero de Personas: " +
        listadoTodas.getListaPersonas().size());
        List<Persona> listaPersonas = new ArrayList<Persona>();
        listaPersonas = listadoTodas.getListaPersonas();
        Iterator iterador = listaPersonas.listIterator();
        while( iterador.hasNext() ) {
           Persona p = (Persona) iterador.next();
           System.out.printf("Nombre: %s, edad: %d %n",
           p.getNombre(), p.getEdad());
        }
        System.out.println("Fin de listado .....");
    } //fin main
}//fin LeerPersonas
```

EJERCICIOS

31. Mostrar la información contenida en el documento clase.xml

32. Muestra el contenido de los coches del archivo concesionario.xml

33. Crea un documento XML con la siguiente estructura:

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<cars>
   <supercars company="Ferrari">
      <carname type="formula one">Ferrari 101</carname>
      <carname type="sports">Ferrari 202</carname>
   </supercars>
</cars>
```

34. Crea un programa que guarde el nombre y apellidos una serie de personas almacenadas en un array de personas.

35. Completa el ejercicio anterior leyendo las personas almacenadas en el fichero creado en el ejercicio anterior.

36. Utiliza SAX para visualizar el contenido del fichero Empleados.xml.

37. Introduce una lista de objetos CuentaBanco en un fichero llamado CuentasBanco.xml.

```
public class CuentaBanco {
   //Miembros dato
   private String nombreTitular;
   private String numCuenta;
   private double saldo;
   //Constructores
   public CuentaBanco(String nombreTitular,String numCuenta,double saldo){
      this.nombreTitular = nombreTitular;
      this.numCuenta = numCuenta;
      this.saldo = saldo;
   }
   public CuentaBanco(){
   }
   //Metodos set
   public void setNombreTitular(String nombreTitular){
      this.nombreTitular = nombreTitular;
   }
   public void setNumCuenta(String numCuenta){
      this.numCuenta = numCuenta;
   }
   //Metodos get
   public String getNombreTitular(){
      return nombreTitular;
   }
   public String getNumCuenta(){
      return numCuenta;
   }
   public double getSaldo(){
      return saldo;
   }
}
```

38. Muestra el contenido del fichero CuentasBanco.xml.

JSON



Douglas Crockford. ***JSON vector logo*** (Dominio público)

JSON o JavaScript Object Notation (notación de objeto de JavaScript, en castellano) es un formato de texto pensado para el intercambio de datos. Debido a su amplia adopción como alternativa a XML, se considera un formato independiente del lenguaje. Inicialmente se hizo muy popular como alternativa a XML en AJAX, puesto que se puede analizar fácilmente usando la función `eval()`, aunque hoy en día, por seguridad, se recomienda utilizar su propio parser.

Al ser JSON un formato muy extendido para el intercambio de datos, se han desarrollado APIs para muchos lenguajes. Y debido a que la sintaxis de JSON es más compacta y más fácil de leer y escribir, lo que le hace más rápido de transmitir y procesar, está reemplazando a XML en muchas aplicaciones. Sin embargo, JSON es menos seguro y menos flexible que XML, por lo que para ciertos usos se sigue usando aún XML.

Generar y transformar ficheros JSON desde Java

Para generar y trabajar documentos JSON desde Java se necesita utilizar alguna librería externa. Existen muchas, y en este caso, vamos a utilizar GSON. Para incluir la librería, lo habitual sería utilizar Maven (u otro gestor de dependencias), como veremos más adelante en el curso. Para Maven, necesitas añadir esta dependencia:

```
<dependency>
    <groupId>com.google.code.gson</groupId>
    <artifactId>gson</artifactId>
    <version>2.13.2</version>
</dependency>
```

Pero, de momento, te puedes descargar GSON de **gson-2.13.2.jar**, e incluirlo en tu proyecto. Para VS Code, crea una carpeta llamada `lib` en la raíz de tu proyecto y coloca ahí el archivo descargado.

Configúralo como **ajuste del espacio de trabajo** para que solo afecte a este proyecto. VS Code guarda estos ajustes en `.vscode/settings.json`, dentro de la carpeta raíz que tienes abierta.

i. Abre en VS Code la carpeta raíz del proyecto: **Archivo** → **Abrir carpeta...**.
ii. Pulsa `Ctrl+Shift+P`.
iii. Escribe y selecciona **Preferences: Open Workspace Settings (JSON)** (en español puede aparecer como **Preferencias: Abrir configuración del área de trabajo (JSON)**).
iv. Se abrirá `settings.json`. Añade esta configuración:

```
{
    "java.project.sourcePaths": ["."],
    "java.project.referencedLibraries": [
        "lib/**/*.jar"
    ]
}
```

Ese patrón le indica a la extensión de Java que añada los archivos `.jar` que encuentre dentro de `lib` al proyecto, que debería quedar más o menos así:

```
Clase/
├── .vscode/
│   └── settings.json
├── lib/
│   └── gson-2.13.2.jar
└── Fichero/
    ├── GeneradorJSON.java
    └── Persona.java
```

En los siguientes ejemplos se lee y escribe una cadena de texto con el documento JSON. Generando JSON desde Java:

```
import com.google.gson.Gson;

public class GeneradorJSON {
    public static void main(String args[]) {
        Persona p1 = new Persona("Miguel", 46);
        Gson gson = new Gson();
        String representacionJSON = gson.toJson(p1);
        System.out.println(representacionJSON);
    }
```

Generando JSON formateado desde Java:

```
import com.google.gson.Gson;
import com.google.gson.GsonBuilder;

public class GeneradorJSON_Formateado {
    public static void main(String args[]) {
        Persona p1 = new Persona("Miguel", 46);
        final Gson gson = new GsonBuilder().setPrettyPrinting().create();
        String representacionJSON = gson.toJson(p1);
        System.out.println(representacionJSON);
    }
}
```

Transformando JSON a objetos Java:

```
import com.google.gson.Gson;
import com.google.gson.reflect.TypeToken;
import java.lang.reflect.Type;
import java.util.List;

public class GeneradorJSON_Objetos {
    public static void main(String args[]) {
        String persona1JSON = "{\"nombre\":\"Miguel\",\"id\":46}";
        String persona2JSON = "{\"nombre\":\"Luisa\",\"id\":55}";
        String personasJSON = "[" + persona1JSON + "," + persona2JSON + "]";
        Gson gson = new Gson();
        Type tipoListaPersona = new TypeToken<List<Persona>>() {
        }.getType();
        List<Persona> personas = gson.fromJson(personasJSON, tipoListaPersona);
        System.out.println("Hay " + personas.size() + " personas.");
    }
}
```

Escribiendo un JSON formateado desde Java:

```
import com.google.gson.Gson;
import com.google.gson.GsonBuilder;
import java.io.FileWriter;
import java.io.IOException;
import java.io.Writer;

public class GenerarFicheroJSONBonito {
    public static void main(String args[]) throws IOException {
        Persona p = new Persona("Miguel", 46);
        Writer writer = new FileWriter("Fichero/persona.json");
        Gson gson = new GsonBuilder().setPrettyPrinting().create();
        gson.toJson(p, writer);
        writer.close();
    }
}
```

Escribiendo un JSON formateado desde Java con varios elementos:

```
import com.google.gson.Gson;
import com.google.gson.GsonBuilder;
import java.io.FileWriter;
import java.io.IOException;
import java.io.Writer;
import java.util.Arrays;
import java.util.List;
/*
Si se escriben dos objetos JSON seguidos en el mismo archivo se produce algo como {"nombre":"Miguel","edad":46}{"nombre":"Rosa","edad":56},
que no es un único documento JSON válido.
Guarda ambos objetos en una lista y serializa la lista: Gson la representa como un array JSON.
*/

public class GenerarFicheroJSONBonito2 {
    public static void main(String[] args) {
        Persona p1 = new Persona("Miguel", 46);
        Persona p2 = new Persona("Rosa", 56);

        List<Persona> personas = Arrays.asList(p1, p2);
        Gson gson = new GsonBuilder().setPrettyPrinting().create();

        try (Writer writer = new FileWriter("Fichero/personas.json")) {
            gson.toJson(personas, writer);
        } catch (IOException e) {
            System.err.println("No se pudo escribir el fichero: " + e.getMessage());
        }
    }
}
```

Leyendo un fichero JSON desde Java:

```
import com.google.gson.Gson;
import com.google.gson.GsonBuilder;
import java.io.BufferedReader;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.InputStream;
import java.io.InputStreamReader;

public class LeerJSONDesdeUnFichero {
    public static void main(String args[]) throws FileNotFoundException {
        Gson gson = new GsonBuilder().setDateFormat("dd/MM/yyyy").create();
        InputStream is = new FileInputStream("Fichero/persona.json");
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(is));
        Persona p = gson.fromJson(bufferedReader, Persona.class);
        System.out.println("La persona se llama: " + p.getNombre());
    }
}
```

Leyendo un fichero JSON con varios elementos desde Java:

```
import com.google.gson.Gson;
import com.google.gson.GsonBuilder;
import com.google.gson.reflect.TypeToken;
import java.io.BufferedReader;
import java.io.FileInputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.lang.reflect.Type;
import java.util.List;

public class LeerJSONDesdeUnFichero2 {
    public static void main(String[] args) {
        Gson gson = new GsonBuilder()
                .setDateFormat("dd/MM/yyyy")
                .create();

        Type tipoListaPersonas = new TypeToken<List<Persona>>() {
        }.getType();

        try (
                InputStream is = new FileInputStream("Fichero/personas.json");
                BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(is))) {
            List<Persona> personas = gson.fromJson(bufferedReader, tipoListaPersonas);

            if (personas == null || personas.isEmpty()) {
                System.out.println("El fichero no contiene personas.");
                return;
            }

            for (Persona persona : personas) {
                System.out.println("La persona se llama: " + persona.getNombre());
            }

        } catch (IOException e) {
            System.err.println("No se pudo leer el fichero: " + e.getMessage());
        }
    }
}
```